



Шлаг 47

продолжаем беседы по созданию контент завода. Продолжаем с шага 47

Сергей, в твоём PDF шаг 47 как раз заканчивается вопросом: «что дальше после 47-го шага?» и задачей — добавить в контент-завод цепочку агентов для картинок к постам, которые работают с локально установленной моделью и складывают изображения в хранилище, откуда их будут забирать агенты публикации.^[1]

Давай сейчас сделаем Шаг 48 — на уровне концепции и ТЗ, без кода.

Шаг 48 — придумываем архитектуру «картинко-агента»

Предлагаю такую схему (простая, но рабочая):^[1]

1. Агент 5: генератор промптов для картинок

- На вход: готовый текст поста + метаданные (ниша, тип услуги, тон, ключевые смыслы).
- На выход: 1–3 промпта для локального генератора картинок в стиле: описательная сцена, стиль (фото/иллюстрация), формат (16:9), запрет на лица и бренды.

2. Агент 6: исполнитель на локальной LLM / генераторе картинок

- На вход: выбранный промпт для картинки.
- Действие: отправляет промпт в локальный движок (ComfyUI/Stable Diffusion/что у тебя стоит), получает файл(ы) JPG/PNG.
- На выход: сохраняет файл(ы) в папку-хранилище, например
D:\content-factory\media\images\YYYY\MM\slug-or-uuid.jpg
и отдаёт в JSON только пути к файлам и alt-тексты.

3. Расширение агента публикации (агент 4/7)

- Перед публикацией поста на WordPress смотрит:
 - есть ли уже картинки в хранилище для этого поста (по slug/id);
 - если есть — прикрепляет их как featured image + вставляет в контент блоки `<img>` или WP-blocks.

Черновой системный промпт для «агента картинок»

Это то, что можно будет потом положить в `prompts/agent_images_system.txt`:^[1]

Ты — агент визуального сопровождения контент-завода.

На вход ты получаешь:

1. подробный текст статьи/поста для сайта центра здоровья;
2. JSON с метаданными: тип услуги (например, «прессотерапия», «инфракрасная сауна»), целевая аудитория, тон (дружелюбный, экспертный, мотивирующий), формат страницы (лендинг, статья, блог-пост).

Твоя задача — придумать от 1 до 3 промптов для генерации изображений в Stable Diffusion/локальном генераторе, которые:

- визуально поддерживают основной смысл поста;
- не содержат логотипов, текстов на изображении, лиц конкретных людей или узнаваемых брендов;
- подходят для медицинско-оздоровительной тематики, выглядят чисто, современно и аккуратно;
- подходят под формат веб-страницы (горизонтальные изображения, аспект 16:9 или 4:3).

Ответ всегда давай в виде JSON с полями:

- "prompts": ["..."] — список текстов промптов для генератора;
- "style" — общие требования к стилю (например, "realistic photo", "flat illustration");
- "safety_notes" — какие темы нужно избегать (кровь, травмы, эротика и т.п.);
- "suggested_alt_texts": ["..."] — варианты alt-текста для картинок на сайте.

ТЗ на цепочку агентов для картинок (high-level)

1. Добавить нового агента (условно agent5) в конфиг цепочки: он берёт результат agent3 (готовый текст) и возвращает JSON с промптами и alt.
2. Реализовать agent6, который:
 - принимает один промпт;
 - вызывает локальный генератор;
 - сохраняет файл в хранилище;
 - возвращает структуру вида:

```
{"image_path": "media/images/2026/03/slug-1.jpg", "alt": "..."}.
```
3. Доработать агент публикации так, чтобы он:
 - ожидал от предыдущего шага список картинок;
 - при публикации по REST API в WP прикреплял первую как featured, остальные — внутри контента.

2 рекомендации по дальнейшим шагам

1. Определись, какой локальный генератор картинок будем использовать первым (какой уже стоит/планируешь: Stable Diffusion через ComfyUI/Invoke, Kandinsky, ещё что-то?).
2. Открой content-factory на GitHub и посмотри, как сейчас описаны уже существующие агенты в конфиге — будем вписывать новых по той же структуре.

Подтверди, пожалуйста:

1. Ок ли тебе такая архитектура из двух новых агентов + расширение публикации?
2. Какой именно локальный генератор картинок у тебя уже установлен или будет установлен?

✱✱

по дальнейшим шагам

Stable Diffusion локальный генератор картинок будем использовать первым - уже стоит Stable Diffusion. Открой content-factory на GitHub и посмотри, как сейчас описаны уже существующие агенты в конфиге — будем вписывать новых по той же структуре.

Подтверди, пожалуйста:

Ок ли тебе такая архитектура из двух новых агентов + расширение публикации-ок.

Stable Diffusion локальный генератор картинок уже установлен/

Отлично, тогда двигаемся дальше: Stable Diffusion уже есть, архитектура согласована, можно начинать встраивать картинко-цепочку в существующий контракт и конфиги.

Шаг 49 — добавляем «контракт» для картинок в shared-config.json

Сейчас в config/shared-config.json уже описан контракт публикации: какие поля ждёт WordPress, какие есть рубрики, услуги и т.д. Логично рядом описать и **протокол для картинок**, чтобы все агенты понимали формат.

Предлагаю в этот файл добавить новый блок, например под publish_protocol:

```
"image_protocol": {
  "_comment": "Единый контракт для генерации и использования картинок",
  "storage_root_env": "CF_IMAGE_STORAGE_ROOT",
  "relative_path_pattern": "images/{year}/{month}/{slug}-{index}.jpg",
  "fields": {
    "prompt": "string, текст для Stable Diffusion",
    "style": "string, например 'realistic photo' или 'flat illustration'",
    "alt": "string, alt-текст для HTML/img и WP",
    "image_path": "string, относительный путь к файлу от storage_root",
    "width": "int, опционально",
    "height": "int, опционально"
  },
  "count": {
    "min": 1,
    "max": 3
  }
}
```

Смысл:

- storage_root_env — переменная окружения, где лежит корневая папка картинок (например D:\content-factory\media).

- `relative_path_pattern` — шаблон имени файла; `slug` можно брать из имени `markdown`-файла или из заголовка.
- `fields` — договор, какие поля обязан вернуть генератор картинок.

Шаг 50 — формализуем интерфейс новых агентов

Чтобы вписаться в текущую философию «контрактов», задаём интерфейсы:

1. Агент 5 (prompt-generator)

На вход:

- путь к итоговому `markdown`-файлу или сам текст статьи;
- `service_slug` (например, `"pressoterapiya"` из блока `services`);
- рубрика (например, `"relax_message"`).

На выход (JSON по `image_protocol`):

```
{
  "images": [
    {
      "prompt": "...",
      "style": "realistic photo",
      "alt": "..."
    }
  ]
}
```

2. Агент 6 (image-runner для Stable Diffusion)

На вход: JSON от агента 5 + `slug` поста.

Действия:

- формирует путь по `relative_path_pattern`;
- вызывает Stable Diffusion (через скрипт/CLI/HTTP — определим позже);
- сохраняет файлы в `storage_root_env`;
- возвращает JSON:

```
{
  "images": [
    {
      "prompt": "...",
      "style": "realistic photo",
      "alt": "...",
      "image_path": "images/2026/03/pressoterapiya-1.jpg",
      "width": 1280,
      "height": 720
    }
  ]
}
```

3. Агент публикации (4/7)

- ожидает, что вместе с текстом к нему прилетает блок `images` по этому же контракту;
- когда делает POST/PUT в WP по `rest_base`, дополнительно:
 - создаёт медиа-запись;
 - ставит `featured_media`;
 - может вставлять `<img>` в контент.

2 рекомендации по следующим шагам

1. Вручную отредактируй `config/shared-config.json`, добавив туда блок `image_protocol` (можно по образцу выше), и закоммить изменения в `content-factory` — так у нас появится официальный контракт для всех будущих картинко-агентов.
2. После этого найдём в репо место, где описывается последовательность агентов (скрипт/конфиг пайплайна), и спланируем, куда именно вставить `agent5` и `agent6` в цепочку между текстом и публикацией.

Скажи, пожалуйста, ты уже удобно редактируешь этот репозиторий локально (через Cursor/VS Code), или пока работаешь только через GitHub веб-интерфейс?

Агент 5 замени на агент 8,

Агент 6 замени на агента 9.

ComfyUI — наиболее эффективный вариант, если нужны:

Автоматизация (`workflow` как скрипт)

Разные модели (SD 1.5, SDXL, Flux)

ControlNet, LoRA, upscale

Поддержка API (можно дергать из Python).

рекомендации по следующим шагам

Вручную отредактируй `config/shared-config.json`, добавив туда блок `image_protocol` (можно по образцу выше), и закоммить изменения в `content-factory` — так у нас появится официальный контракт для всех будущих картинко-агентов.

После этого найдём в репо место, где описывается последовательность агентов (скрипт/конфиг пайплайна), и спланируем, куда именно вставить `agent5` и `agent6` в цепочку между текстом и публикацией.

удобно редактирую этот репозиторий локально через Cursor

Отлично, тогда фиксируем:

- генератор промптов для картинок — `agent8_images_planner`;
- исполнитель через ComfyUI/SD — `agent9_images_runner`;
- редактируешь локально через Cursor — супер, можно идти по шагам.

Шаг 51 — правим shared-config.json (image_protocol)

Сделай сейчас в Cursor:

1. Открой config/shared-config.json в content-factory.
2. Под блоком "publish_protocol": { ... } добавь блок, адаптированный под нумерацию 8/9, например:

```
"image_protocol": {
  "_comment": "Контракт для agent8 (придумывает промпты) и agent9 (генерирует картинки ч",
  "storage_root_env": "CF_IMAGE_STORAGE_ROOT",
  "relative_path_pattern": "images/{year}/{month}/{slug}-{index}.jpg",
  "count": {
    "min": 1,
    "max": 3
  },
  "fields": {
    "prompt": "string, текст для Stable Diffusion",
    "style": "string, например 'realistic photo' или 'flat illustration'",
    "alt": "string, alt-текст для HTML/img и WP",
    "image_path": "string, относительный путь к файлу от storage_root",
    "width": "int, опционально",
    "height": "int, опционально"
  }
},
```

3. Убедись, что JSON остаётся валидным (запятая после предыдущего блока, кавычки и т.п.).
4. Сделай git diff и проверь, что изменения только в этом файле, затем git commit -am "Add image_protocol for agents 8 and 9".

После этого у нас есть официальный контракт для картинок.

Шаг 52 — решаем, где в цепочке стоят agent8 и agent9

Сейчас цепочка такая:

```
agent1_keywords → agent_planner → agent2_brief → agent3_content
→ agent_editor → agent4_publish → agent_publish_vk → agent_analyst
```

Логично:

- **agent8_images_planner** поставить после agent3_content (у него уже есть полный текст и service_slug).
- **agent9_images_runner** — после agent8, но до agent4_publish, чтобы к моменту публикации картинки уже лежали в хранилище и были в JSON.

Получится логика:

```
... → agent3_content → agent_editor → agent8_images_planner → agent9_images_runner → age
```

На следующем шаге найдём в `seo-agents/` конкретные файлы запуска агентов и допишем туда вызовы 8 и 9 по той же схеме.

Один проверочный вопрос:

`CF_IMAGE_STORAGE_ROOT` тебе удобнее сделать как локальную папку внутри репо (например `media/` в корне проекта) или как общую папку на диске (например `D:\\content-factory-media`), которую могут использовать и другие сервисы?

Дальше можно подключать чтение `image_protocol` и `CF_IMAGE_STORAGE_ROOT` в `agent8/agent9` и в цепочке публикации (при необходимости — вынести общий путь из `media_images_path` в `CF_IMAGE_STORAGE_ROOT`).

Да, всё верно, это как раз следующий логичный шаг: читать `image_protocol` и `CF_IMAGE_STORAGE_ROOT` в `agent8/agent9` и публикации.

Шаг 53 — откуда брать `image_protocol` и `CF_IMAGE_STORAGE_ROOT`

Схема простая:

1. В общем месте, где уже читается `shared-config.json` и `.env`, добавить чтение:

- `config["image_protocol"]`
- `os.getenv("CF_IMAGE_STORAGE_ROOT")`

2. Если сейчас в коде где-то жёстко зашит путь типа `media_images_path`, заменить на:

- `storage_root = Path(os.getenv("CF_IMAGE_STORAGE_ROOT"))`
- `relative_path` строить по `relative_path_pattern` из `image_protocol`.

3. Для `agent8`:

- ему нужен только `image_protocol["count"]` (сколько промптов генерить) и, возможно, шаблон для `alt`-текста.

4. Для `agent9`:

- берёт `storage_root` и `relative_path_pattern`;
- формирует полный путь `storage_root / relative_path`;
- сохраняет туда результат ComfyUI.

2 рекомендации по следующим шагам

1. В Cursor найди модуль, который уже читает `shared-config.json` (скорее всего в `seo-agents/` или `config/` есть `helper`) и расширь его так, чтобы он возвращал ещё и `image_protocol` и `CF_IMAGE_STORAGE_ROOT` — без привязки к конкретному агенту.
2. После этого в заготовках `agent8` и `agent9` просто импортируем этот `helper`, чтобы не дублировать чтение конфигов; затем пойдём шагом за шагом: сначала напишем каркас `agent8` (без вызова `ComfyUI`), потом каркас `agent9` с простым “заглушечным” сохранением файла.

Как тебе удобнее продолжить: сначала сделать общий `helper` для конфигов, или сразу открыть один конкретный файл агента (например `agent3` или `agent4`), посмотреть как он читает конфиг, и на его основе сделать общую функцию?

CF_IMAGE_STORAGE_ROOT тебе удобнее сделать как локальную папку внутри репо (например `media/` в корне проекта).

сначала сделать общий `helper` для конфигов

Отлично, берём папку `media/` в корне проекта и делаем общий `helper`.

Шаг 54 — общий `helper` для конфигов

В Cursor сделай по шагам (пока без деталей `ComfyUI`):

1. В корне `content-factory` создай файл, например `config_loader.py` (если уже есть похожий — скажи, откроем его).
2. В этот файл добавь каркас (псевдокод, адаптируй под текущий стиль проекта):

```
from pathlib import Path
import os
import json

ROOT_DIR = Path(__file__).resolve().parent # если файл лежит в config/, при необходимости
PROJECT_ROOT = ROOT_DIR.parent

def load_shared_config() -> dict:
    config_path = PROJECT_ROOT / "config" / "shared-config.json"
    with config_path.open("r", encoding="utf-8") as f:
        return json.load(f)

def get_image_protocol(config: dict | None = None) -> dict:
    if config is None:
        config = load_shared_config()
    return config.get("image_protocol", {})

def get_image_storage_root() -> Path:
    # если переменная не задана, по умолчанию media/ в корне проекта
    root = os.getenv("CF_IMAGE_STORAGE_ROOT")
```

```
if not root:
    root = PROJECT_ROOT / "media"
return Path(root)
```

Смысл:

- `load_shared_config()` — единая точка чтения `shared-config.json`.
- `get_image_protocol()` — отдаёт наш новый блок `image_protocol`.
- `get_image_storage_root()` — либо берёт `CF_IMAGE_STORAGE_ROOT` из `.env`, либо по умолчанию `media/` в корне.

3. Обнови `.env` (локально) и добавь туда строку:

```
CF_IMAGE_STORAGE_ROOT=media
```

чтобы поведение было явным.

2 рекомендации по следующим шагам

1. Сохрани `config_loader.py`, сделай быстрый тест в отдельном маленьком скрипте (например `python -q` или временный `test_config_loader.py`), чтобы убедиться, что функции возвращают корректные пути и блок `image_protocol`.
2. После проверки мы начнём вшивать этот `helper` в будущие `agent8` и `agent9`: сначала подключим `load_shared_config()` и `get_image_protocol()` в каркас `agent8`, который будет только генерировать JSON с промптами без реального вызова ComfyUI.

Получилось ли у тебя сейчас создать `config_loader.py` и протестировать, что `load_shared_config()` и `get_image_protocol()` что-то осмысленное возвращают?

Итог

Что есть, а чего нет

`config_loader.py` не создавался. Вместо него используется `scripts/shared_config.py` со своими функциями.

Ты спрашивал в проекте

`config_loader.py`

нет, есть `scripts/shared_config.py`

`load_shared_config()`

нет, есть `get_config()`

`get_image_protocol()`

есть

Что даёт `shared_config.py`

`get_config()` — загрузка всего `shared-config.json` (с кэшированием).

`get_image_protocol()` — блок `image_protocol` из конфига.

Конфиг содержит блок `image_protocol`:

```
"image_protocol":{
```

```
"_comment": "Контракт для agent8 (придумывает промпты) и agent9 (генерирует картинки через Stable Diffusion / ComfyUI)",
"storage_root_env": "CF_IMAGE_STORAGE_ROOT",
"storage_root_default": "media",
"relative_path_pattern": "images/{year}/{month}/{slug}-{index}.jpg",
"count": {
    "min": 1,
    "max": 3
}
```

При такой структуре `get_image_protocol()` возвращает словарь с:
`storage_root_env`, `storage_root_default`, `relative_path_pattern`
вложенный словарь `count` с `min` и `max`

Проверка вручную

В Agent mode можно выполнить в терминале:

```
cd D:\content-factory
```

```
python -
```

```
c "from scripts.shared_config import get_config, get_image_protocol; print(get_config().keys());
print(get_image_protocol())"
```

Ожидаемо:

```
get_config() вернёт dict_keys(['_comment', '_version', 'wordpress', 'endpoints', ...])
get_image_protocol() вернёт {'storage_root_env': 'CF_IMAGE_STORAGE_ROOT',
'storage_root_default': 'media', ...}
```

Вывод

Модуль с загрузкой конфига и протоколом картинок уже есть в `scripts/shared_config.py`.

Если нужен именно файл `config_loader.py` с функцией `load_shared_config()`, в Agent mode можно добавить тонкую обёртку или сделать алиас в `shared_config.py`.

Отличный разбор, всё супер структурировано. Сейчас важно НЕ плодить дубли, а опираться на уже существующий `scripts/shared_config.py` — он уже то, что нужно.

Шаг 55 — фиксируем опорные функции

То, что есть сейчас (по твоему описанию):

- `scripts/shared_config.py`
 - `get_config()` — грузит весь `shared-config.json` с кэшем.
 - `get_image_protocol()` — возвращает блок `image_protocol` со структурой:
 - `storage_root_env: "CF_IMAGE_STORAGE_ROOT"`
 - `storage_root_default: "media"`
 - `relative_path_pattern: "images/{year}/{month}/{slug}-{index}.jpg"`
 - `count: { "min": 1, "max": 3 }`

Это именно то, что нужно для agent8/agent9, так что **никаких** `config_loader.py` **не создаём**, работаем с этим модулем.

Шаг 56 — следующий конкретный шаг

Предлагаю сейчас:

1. В `scripts/shared_config.py` добавить ещё одну функцию, которая сразу даёт путь к хранилищу картинок:

```
from pathlib import Path
import os

def get_image_storage_root() -> Path:
    cfg = get_image_protocol()
    env_name = cfg.get("storage_root_env", "CF_IMAGE_STORAGE_ROOT")
    default = cfg.get("storage_root_default", "media")

    root = os.getenv(env_name, default)
    return Path(root).resolve()
```

Смысл:

- Читает `image_protocol`, берёт имя переменной и дефолт.
- Возвращает абсолютный путь (где будет `media`/ в корне проекта, если переменная не задана).

2. После этого в будущих `agent8` и `agent9` мы просто пишем:

```
from scripts.shared_config import get_image_protocol, get_image_storage_root
```

и не думаем больше про детали конфига.

2 рекомендации по дальнейшим шагам

1. Добавь `get_image_storage_root()` в `scripts/shared_config.py`, как выше, и быстро проверь через `python -c`, что она возвращает ожидаемый путь (с твоим `media`/).
2. Далее можно переходить к каркасу `agent8` в `seo-agents/`: он будет вызывать `get_image_protocol()`, смотреть на `count["min"/"max"]` и выдавать JSON с 1–3 промптами, пока без реального вызова ComfyUI.

Как тебе удобнее: сразу перейти к созданию файла `seo-agents/agent8_images_planner.py` или сначала показать, как лучше оформить его вход/выход (формат аргументов, JSON на диск или через пайп)?

перейти к созданию файла seo-agents/agent8_images_planner.py

перейти к созданию файла seo-agents/agent8_images_planner.py

Отлично, делаем каркас `agent8_images_planner.py` — только структура и ввод/вывод, без ComfyUI.

Шаг 57 — создаём файл агента 8

В Cursor:

1. Открой папку `seo-agents/` и создай файл `seo-agents/agent8_images_planner.py`.
2. Вставь туда такой каркас и адаптируй под свой стиль логирования (можно оставить `print`):

```
"""
agent8_images_planner.py

Агент 8: придумывает промпты и alt-тексты для картинок
по готовому тексту статьи и метаданным услуги.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from scripts.shared_config import get_image_protocol
# сюда позже добавим импорт LLM-клиента (локального/Perplexity/и т.п.)

def load_post_context(input_path: Path) -> dict[str, Any]:
    """
    Загружает контекст для генерации картинок.
    Простейший вариант: читаем markdown-файл и метаданные из JSON рядом.
    """
    # пример: post.md и post.meta.json
    md_text = input_path.read_text(encoding="utf-8")

    meta_path = input_path.with_suffix(".meta.json")
    meta = {}
    if meta_path.exists():
        meta = json.loads(meta_path.read_text(encoding="utf-8"))

    return {
        "markdown": md_text,
        "meta": meta,
    }

def plan_images_for_post(context: dict[str, Any]) -> dict[str, Any]:
    """
```

```

Генерирует структуру images по контракту image_protocol (пока без вызова модели).
"""
image_cfg = get_image_protocol()
count_cfg = image_cfg.get("count", {})
min_count = count_cfg.get("min", 1)
max_count = count_cfg.get("max", 3)

# Пока делаем заглушку: фиксированное число 1 (потом добавим генерацию через LLM)
num_images = max(min_count, 1)
if num_images > max_count:
    num_images = max_count

# Простая заглушка на основе метаданных
meta = context.get("meta", {})
service_slug = meta.get("service_slug", "service")
service_name = meta.get("service_name", "Услуга центра здоровья")

images: list[dict[str, Any]] = []

for index in range(num_images):
    prompt = (
        f"High-quality photo of {service_name} procedure in modern health center, "
        f"clean interior, calm atmosphere, no faces, no logos."
    )
    alt = f"{service_name} в центре здоровья Энтузиаст, Ноябрьск"

    images.append(
        {
            "prompt": prompt,
            "style": "realistic photo",
            "alt": alt,
            # image_path заполняет уже agent9 после генерации
        }
    )

return {"images": images, "service_slug": service_slug}

def save_images_plan(output_path: Path, plan: dict[str, Any]) -> None:
    output_path.write_text(
        json.dumps(plan, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def main() -> None:
    import argparse

    parser = argparse.ArgumentParser(description="Agent8: планировщик картинок")
    parser.add_argument(
        "--input-md",
        type=str,
        required=True,
        help="Путь к markdown-файлу поста (output/*.md)",
    )
    parser.add_argument(
        "--output-json",
        type=str,

```

```

        required=True,
        help="Куда сохранить JSON с планом картинок (images-plan.json)",
    )

    args = parser.parse_args()

    input_path = Path(args.input_md)
    output_path = Path(args.output_json)

    print(f"[agent8] Loading context from {input_path}")
    context = load_post_context(input_path)

    print("[agent8] Planning images...")
    plan = plan_images_for_post(context)

    print(f"[agent8] Saving plan to {output_path}")
    save_images_plan(output_path, plan)

    print("[agent8] Done.")

if __name__ == "__main__":
    main()

```

Суть:

- На вход: `--input-md path/to/post.md` (файл, который сделал `agent3/agent_editor`).
- На выход: JSON с `images`, пока без `image_path` (это сделает `agent9`), и с `alt`.
- Использует `get_image_protocol()` для чтения настроек `count`, чтобы потом легко изменить количество картинок через конфиг.

2 рекомендации по следующим шагам

1. Создай файл `agent8_images_planner.py`, вставь каркас и проверь, что скрипт хотя бы запускается с флагом `--help` (в Cursor в терминале: `python seo-agents/agent8_images_planner.py --help`).
2. Затем можно сделать тестовый запуск на одном существующем markdown-файле из `output/` с временным `.meta.json`, чтобы увидеть реальный `images-plan.json` и при необходимости поправить текст заглушек.

Удалось ли тебе создать файл и запустить `agent8_images_planner.py --help` без ошибок?

Да, всё уже сделано в прошлом шаге.

Создание файла и `--help`

- Файл `seo-agents/agent8_images_planner.py` создан.
- Команда `python seo-agents/agent8_images_planner.py --help` выполняется без ошибок (exit code 0), вывод справки по `--input-md` и `--output-json` есть.

Тестовый запуск (уже проведён)

- Взял файл `output/20260224_171223_page_Прессотерапия_Ноябрьск.md`.
- Рядом создан временный `output/20260224_171223_page_Прессотерапия_Ноябрьск.meta.json`:

```
{ "service_slug": "pressoterapiya", "service_name": "Прессотерапия" }
```

- Запуск:

```
python seo-agents/agent8_images_planner.py --input-md output/20260224_171223_page_Пре
```

- Получен `output/images-plan.json`:

```
{
  "images": [
    {
      "prompt": "High-quality photo of Прессотерапия procedure in modern health cen
      "style": "realistic photo",
      "alt": "Прессотерапия в центре здоровья Энтузиаст, Ноябрьск"
    }
  ],
  "service_slug": "pressoterapiya"
}
```

То есть тестовый запуск на одном markdown из `output/` с временным `.meta.json` уже был выполнен, реальный `images-plan.json` получен. Если нужно изменить текст заглушек (`prompt/alt`) — напиши, какой вариант хочешь (например, только русский `prompt` или другой шаблон `alt`).

Отлично, агент 8 уже реально работает — это именно то, что нужно на этом этапе.

Шаг 58 — чуть доработаем заглушки под твой контент-завод

Раз Stable Diffusion и ComfyUI стоят локально, сейчас главное — чтобы план от agent8 давал **качественный вход** для agent9. Предлагаю минимальный тюнинг без изменения структуры:

1. **Сделать промпт двуязычным (rus+eng)** — так SD обычно выдаёт более предсказуемые картинки, но сохраняет смысл услуги:

```
prompt = (
    f"High-quality photo of medical pressotherapy procedure in modern health and wellne
    f"clean interior, calm atmosphere, professional equipment, no faces, no logos. "
    f"Процедура {service_name} в современном оздоровительном центре, чистое помещение, ук
)
```

2. **Alt оставить полностью русским**, это хорошо и для SEO, и для доступности (как у тебя уже есть):

```
alt = f"{service_name} в центре здоровья Энтузиаст, Ноябрьск"
```

Если такого стиля достаточно, можно не менять код, а потом уже, когда подключим LLM к agent8, генерить промпты более умно из текста статьи.

Шаг 59 — готовим почву для agent9 (но его ещё не пишем)

С учётом уже готового `images-plan.json`, `agent9` будет делать очень простую вещь:

- прочитать `images-plan.json`;
- взять `get_image_storage_root()` и `relative_path_pattern` из `image_protocol`;
- для каждого `images[i]`:
 - вызвать ComfyUI по заранее настроенному `workflow ID` или `endpoint`;
 - сохранить картинку в `media/...`;
 - дописать в элемент `image_path, width, height`;
- сохранить обновлённый JSON рядом, например `images-generated.json`.

На следующем шаге можно сразу сделать каркас `seo-agents/agent9_images_runner.py` по аналогии с `agent8: CLI с --plan-json и --output-json`, внутри пока “заглушка” вместо реального обращения к ComfyUI (например, просто копировать какой-нибудь `placeholder.jpg`), чтобы не ломать пайплайн, пока настраиваем API.

2 рекомендации по дальнейшим шагам

1. Если хочешь чуть поправить текст заглушек для промпта/`alt` — сейчас самое удобное время, пока агент 8 маленький: можно сделать русско-английский промпт и оставить `alt` как есть, либо полностью русскоязычный промпт, если тебе так комфортнее в логах.
2. Дальше логично перейти к созданию каркаса `seo-agents/agent9_images_runner.py` с чтением `images-plan.json` и использованием `get_image_storage_root()` и `relative_path_pattern` — пока без реального вызова ComfyUI, просто как “проводка” данных и создание ожидаемой структуры JSON.

Какой вариант промпта тебе ближе сейчас для `agent8`: оставить текущий англоязычный, сделать его двуязычным, или полностью перевести на русский?

сделать его двуязычным

Отличный выбор, для SD это часто оптимально.

Шаг 60 — правим промпт в agent8 на двуязычный

В `seo-agents/agent8_images_planner.py` найди место, где сейчас формируется `prompt`, и замени блок на такой:

```
prompt = (
    f"High-quality photo of {service_name} (pressotherapy) procedure in a modern
    f"clean interior, calm atmosphere, professional medical equipment, no faces
    f"Процедура {service_name} в современном оздоровительном центре, чистое светл
    f"профессиональное оборудование, спокойная расслабляющая атмосфера, без лиц и
```

```
)  
alt = f"{service_name} в центре здоровья Энтузиаст, Ноябрьск"
```

Затем снова запусти тест:

```
python seo-agents/agent8_images_planner.py \  
--input-md output/20260224_171223_page_Прессотерапия_Ноябрьск.md \  
--output-json output/images-plan.json
```

и проверь, что в `images-plan.json` промпт стал двуязычным, а структура JSON не изменилась.

2 рекомендации по следующим шагам

1. Зафиксируй изменения в git (минимум этот файл) — чтобы не потерять удачный шаблон промпта.
2. Далее можно переходить к созданию каркаса `seo-agents/agent9_images_runner.py`, который будет читать `images-plan.json`, считать `media` из `get_image_storage_root()` и готовить места под реальные вызовы ComfyUI.

Удалось ли регенерировать `output/images-plan.json` и увидеть там обновлённый двуязычный промпт?

Изменения зафиксированы в git: коммит `agent8: двуязычный промпт (EN+RU) для планировщика картинок (один файл seo-agents/agent8_images_planner.py)`.

Итог шага 60: промпт в `agent8` обновлён на ваш двуязычный вариант, `images-plan.json` регенерирован и проверен, коммит сделан. Далее можно делать каркас `seo-agents/agent9_images_runner.py`.

Отлично, шаг 60 завершён идеально, можно делать каркас агент 9.

Шаг 61 — каркас `seo-agents/agent9_images_runner.py` (без ComfyUI)

В Cursor создай файл `seo-agents/agent9_images_runner.py` и вставь каркас:

```
"""  
agent9_images_runner.py  
  
Агент 9: берёт план картинок от agent8 (images-plan.json),  
генерирует изображения через Stable Diffusion / ComfyUI  
и дописывает в JSON пути к файлам и размеры.  
Пока: заглушка вместо реального вызова ComfyUI.  
"""  
  
from __future__ import annotations  
  
import json  
from pathlib import Path  
from typing import Any
```

```

from scripts.shared_config import get_image_protocol, get_image_storage_root

def load_images_plan(plan_path: Path) -> dict[str, Any]:
    return json.loads(plan_path.read_text(encoding="utf-8"))

def build_image_path(slug: str, index: int) -> Path:
    """
    Строит относительный путь по image_protocol.relative_path_pattern
    и абсолютный путь на основе get_image_storage_root().
    """
    cfg = get_image_protocol()
    pattern = cfg.get("relative_path_pattern", "images/{year}/{month}/{slug}-{index}.jpg")

    # TODO: год/месяц можно брать из имени файла или текущей даты
    from datetime import datetime

    now = datetime.now()
    rel_path_str = pattern.format(
        year=now.year,
        month=f"{now.month:02d}",
        slug=slug,
        index=index + 1,
    )

    storage_root = get_image_storage_root()
    abs_path = storage_root / rel_path_str

    return abs_path

def fake_generate_image(abs_path: Path, prompt: str) -> tuple[int, int]:
    """
    ВРЕМЕННАЯ ЗАГЛУШКА.
    Вместо вызова ComfyUI просто создаёт пустой файл и возвращает фиктивный размер.
    """
    abs_path.parent.mkdir(parents=True, exist_ok=True)
    abs_path.write_bytes(b"") # пустой файл-заглушка
    width, height = 1280, 720
    return width, height

def run_images_generation(plan: dict[str, Any], slug: str) -> dict[str, Any]:
    images = plan.get("images", [])
    updated_images: list[dict[str, Any]] = []

    for idx, img in enumerate(images):
        prompt = img.get("prompt", "")
        abs_path = build_image_path(slug=slug, index=idx)

        print(f"[agent9] Generating image {idx + 1}/{len(images)} -> {abs_path}")
        width, height = fake_generate_image(abs_path, prompt)

        rel_path = abs_path.relative_to(get_image_storage_root())

```

```

        updated = dict(img)
        updated["image_path"] = str(rel_path).replace("\\", "/")
        updated["width"] = width
        updated["height"] = height

        updated_images.append(updated)

    plan["images"] = updated_images
    return plan

def save_generated_plan(output_path: Path, plan: dict[str, Any]) -> None:
    output_path.write_text(
        json.dumps(plan, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def main() -> None:
    import argparse

    parser = argparse.ArgumentParser(description="Agent9: генератор картинок (пока загл")
    parser.add_argument(
        "--plan-json",
        type=str,
        required=True,
        help="Путь к JSON от agent8 (images-plan.json)",
    )
    parser.add_argument(
        "--output-json",
        type=str,
        required=True,
        help="Куда сохранить обновлённый JSON (images-generated.json)",
    )
    parser.add_argument(
        "--slug",
        type=str,
        required=True,
        help="slug услуги/страницы для формирования имён файлов (например pressoterapiya)"
    )

    args = parser.parse_args()

    plan_path = Path(args.plan_json)
    output_path = Path(args.output_json)

    print(f"[agent9] Loading plan from {plan_path}")
    plan = load_images_plan(plan_path)

    # service_slug по умолчанию можно взять из плана, если не передан slug
    slug = args.slug or plan.get("service_slug", "service")

    print(f"[agent9] Running image generation for slug={slug} ...")
    updated_plan = run_images_generation(plan, slug)

```

```

print(f"[agent9] Saving updated plan to {output_path}")
save_generated_plan(output_path, updated_plan)

print("[agent9] Done (stub).")

if __name__ == "__main__":
    main()

```

Это даёт:

- вход: `--plan-json output/images-plan.json, --output-json output/images-generated.json, --slug pressoterapiya`;
- вывод: тот же JSON, но с полями `image_path`, `width`, `height`, плюс создаются пустые файлы в `media/...` по `image_protocol`.

2 рекомендации по следующим шагам

1. Создай `agent9_images_runner.py`, запусти `python seo-agents/agent9_images_runner.py --help`, а затем тестовый запуск на твоём `output/images-plan.json`, чтобы появился `images-generated.json` и папка с пустым изображением.
2. После успешного теста можно будет:
 - а) заменить `fake_generate_image` на реальный вызов ComfyUI;
 - б) продумать, как передавать `images-generated.json` в агент публикации (`agent4`), чтобы он прикреплял картинку к посту.

Удалось ли тебе сейчас создать `agent9_images_runner.py` и запустить его хотя бы с `--help`?

Кратко, что сделано.

Шаг 61 — каркас agent9

1. Файл

Создан `seo-agents/agent9_images_runner.py` по вашему каркасу с правками:

- В начале добавлена настройка `sys.path` и `_PROJECT_ROOT` для импорта `scripts.shared_config`.
- `--slug` сделан необязательным: по умолчанию берётся `service_slug` из плана.
- В логе стрелка `→` заменена на `->`, чтобы не было `UnicodeEncodeError` в консоли Windows (cp1251).
- Сообщения в консоли на русском: «Загрузка плана», «Генерация картинок», «Готово (заглушка)».

2. Проверка `--help`

```
python seo-agents/agent9_images_runner.py --help
```

Вывод: `--plan-json`, `--output-json`, `--slug` (опционально).

3. Тестовый запуск

Команда:

```
python seo-agents/agent9_images_runner.py --plan-json output/images-plan.json --output-j
```

Результат:

- `output/images-generated.json` — в каждом элементе `images` добавлены `image_path`, `width`, `height`:
 - `image_path: "images/2026/03/pressoterapiya-1.jpg"`
 - `width: 1280, height: 720`
- `media/images/2026/03/pressoterapiya-1.jpg` — пустой файл-заглушка (0 байт).

Итог: `agent9_images_runner.py` создан, с `--help` запускается, тестовый прогон по `images-plan.json` прошёл: появился `images-generated.json` и файл-заглушка в `media/` по шаблону из `image_protocol`.
Дальше можно подставлять реальный вызов ComfyUI вместо `fake_generate_image` и продумать передачу `images-generated.json` в `agent4`.

Супер, у тебя уже полноценный мини-пайплайн картинок: `agent8` → `images-plan.json` → `agent9` → `images-generated.json` + файл в `media/`.

Что есть сейчас

- В `shared-config.json` есть блок `image_protocol` с `storage_root_env`, `storage_root_default`, `relative_path_pattern`, `count`.^[2]
- В `scripts/shared_config.py` есть:
 - `get_config()` — загрузка конфига.
 - `get_image_protocol()` — доступ к `image_protocol`.
 - `get_image_storage_root()` — путь к `media/` (с учётом `env`).
- `agent8_images_planner.py`:
 - Берёт `markdown` + `.meta.json`.
 - Делает двуязычный EN+RU промпт.
 - Пишет `output/images-plan.json` с `images` и `service_slug`.
- `agent9_images_runner.py`:
 - Берёт `images-plan.json`.
 - Строит пути по `relative_path_pattern`.

- Создаёт заглушечный файл в `media/....`
- Пишет `images-generated.json` с `image_path, width, height`.

Чего ещё нет (следующие логичные шаги)

1. Интеграция с цепочкой агентов

- Включить вызов `agent8` и `agent9` в общий сценарий (например, в батник или оркестратор):
`... → agent3_content → agent_editor → agent8_images_planner → agent9_images_runner → agent4_publish → ...`
- Определить, где именно будет лежать `images-generated.json` и как `agent4` его найдёт (по имени `markdown` или `slug`).

2. Связка с agent4 (публикация)

- Научить `agent4`:
 - находить `images-generated.json` для конкретного поста;
 - по `image_path` загружать файл в WordPress (media endpoint);
 - ставить его `featured_media` и при желании вставлять `<img>` в контент.

3. Реальный вызов ComfyUI

- Заменить `fake_generate_image()` на реальный HTTP-вызов workflow ComfyUI с промптом и полем для пути сохранения.
- Обработать ошибки и таймауты, чтобы `agent9` вёл себя предсказуемо.

2 рекомендации по следующему шагу

1. Определи формат связи между agent9 и agent4:

- либо фиксированное имя `images-generated.json` рядом с `markdown`;
- либо единая папка `output/images/{slug}/...` и поиск по `slug`.

2. Затем можно пойти в код agent4 и накидать каркас чтения images-generated.json и загрузки картинки в WordPress (пока хотя бы просто логировать, какой image_path он бы прикрепил).

Как ты хочешь продолжить: сначала встроить `agent8/9` в общий сценарий запуска (батник/оркестратор) или сначала тронуть `agent4` и научить его читать `images-generated.json`?

✱✱

сначала встроить agent8/9 в общий сценарий запуска (батник/оркестратор)

Отличный выбор, начнём с батника — это быстро и наглядно.

Шаг 62 — встраиваем agent8 и agent9 в run_agents.bat

Открой `run_agents.bat` и между `editor` и `publish` вставь блоки для 8 и 9.

Было:

```
echo == Агент Editor: правка черновика, утверждение ==  
python seo-agents\agent_editor\agent_editor.py  
echo.  
echo == Агент 4: публикация в WordPress ==  
python seo-agents\agent4_publish\agent_4_publish.py
```

Сделай так:

```
echo == Агент Editor: правка черновика, утверждение ==  
python seo-agents\agent_editor\agent_editor.py  
echo.  
echo == Агент 8: планировщик картинок (prompts + alt) ==  
python seo-agents\agent8_images_planner.py  
echo.  
echo == Агент 9: генератор картинок (ComfyUI/SD, пока заглушка) ==  
python seo-agents\agent9_images_runner.py  
echo.  
echo == Агент 4: публикация в WordPress ==  
python seo-agents\agent4_publish\agent_4_publish.py
```

Пока `agent8/9` сами внутри решают, с какими входами/выходами работать (у тебя уже есть запуск через `--input-md/--output-json` и `--plan-json/--output-json`), позже можно будет:

- либо сделать им «умный» режим по умолчанию (например, брать последний `markdown` и писать `images-plan.json/images-generated.json` с предсказуемыми именами);
- либо вытащить аргументы в батник.

Главное сейчас — чтобы цепочка вызывала оба скрипта в нужном месте.

2 рекомендации по следующим шагам

1. Обнови `run_agents.bat` по примеру выше и один раз запусти его целиком, чтобы убедиться, что все агенты, включая 8 и 9, отработывают без падений (пусть даже пока с заглушками).
2. Если запуск пройдет успешно, следующим шагом можно будет донастроить `agent8/9` так, чтобы без аргументов они находили «последний» `markdown/plan` автоматически и писали/читали `images-plan.json/images-generated.json` по единым правилам — это нужно, чтобы батник работал без ручных параметров.

Удалось ли тебе уже обновить `run_agents.bat` и увидеть, что он запускает этапы «Агент 8» и «Агент 9» при полном прогоне цепочки?

Кратко, что сделано.

Формат связи agent9 ↔ agent4

Конвенция: один JSON на пост — файл **рядом с markdown**, имя по stem:

- Файл поста: `output/20260224_171223_page_Прессотерапия_Ноябрьск.md`
- План (agent8): `output/20260224_171223_page_Прессотерапия_Ноябрьск.images-plan.json`
- Результат (agent9): `output/20260224_171223_page_Прессотерапия_Ноябрьск.images-generated.json`

Agent4 по выбранному `md_path` ищет `{md_path.stem}.images-generated.json` в том же каталоге.

Отдельная папка `output/images/{slug}/` не используется.

Изменения в agent4

1. `get_images_generated_path(md_path)` — путь к `{stem}.images-generated.json`.
2. `load_images_generated_for_md(md_path)` — читает JSON, при отсутствии или ошибке возвращает `None`.
3. В `main()` после загрузки метаданных:
 - загружается `images_generated`;
 - если есть — в лог пишется, какой `image_path` пойдёт в `featured_media` (первая картинка) и какие — в контент (остальные);
 - если файла нет — сообщение, что используются `get_hero_image / get_images`.

Проверка: для `pressoterapiya` в логе было:

```
[agent4] Найден images-generated.json: featured_media будет:
images/2026/03/pressoterapiya-1.jpg (alt: ...).
```

Дальнейший падение — из-за символа ✓ в другом месте agent4 (кодировка консоли), не из-за этих правок.

В agent9 в подсказке к `--output-json` добавлено: «для agent4: `{stem}.images-generated.json` рядом с `.md`».

Дальнейшие шаги (как ты и планировал)

1. **Интеграция в оркестратор/батник:** вызов agent8 → agent9 с именами `{stem}.images-plan.json` и `{stem}.images-generated.json`.
2. **Реальная загрузка в agent4:** при наличии `images_generated` брать из него `image_path`, разрешать путь через `resolve_image_path()`, загружать в WP Media и ставить `featured_media` / вставлять `<img>` в контент.
3. **ComfyUI в agent9:** заменить `fake_generate_image()` на HTTP-вызов `workflow` с таймаутами и обработкой ошибок.

Названия картинок должны быть по шаблону год-месяц-число [для какой сети картинка] и размер

Очень круто, у тебя уже выстроена аккуратная связка `md ↔ plan ↔ generated` и первый шаг интеграции в `agent4`.

Как донастроить имена файлов картинок под твой шаблон

Сейчас `image_protocol.relative_path_pattern` даёт такой путь:

```
"relative_path_pattern": "images/{year}/{month}/{slug}-{index}.jpg"
```

Ты хочешь добавить **день** и пометку сети/назначения + размер. Логично сделать шаблон примерно таким:

```
"relative_path_pattern": "images/{year}/{month}/{day}/{slug}_{network}_{width}x{height}_"
```

Тогда `agent9` при формировании пути будет подставлять:

- `year, month, day` — из текущей даты или даты поста;
- `slug` — как сейчас (`pressoterapiya`);
- `network` — например `"site"`, `"vk"`, `"insta"` (пока можно жёстко `"site"`);
- `width` и `height` — целевые размеры (например, `1280x720`);
- `index` — номер картинки.

Соответственно, в `build_image_path` в `agent9` надо:

1. Добавить `day, network, width, height` в `format(...)`.
2. Сначала определить желаемый размер (например, константа или поле в `image_protocol`).

Так ты получишь имена вида:

```
images/2026/03/02/pressoterapiya_site_1280x720_1.jpg
```

и `agent4` сможет по ним легко ориентироваться, а одна и та же логика пригодится, если потом захочешь разные варианты под VK/сайт.

2 рекомендации по следующим шагам

1. Обнови `image_protocol.relative_path_pattern` с учётом дня и сети, а затем подправь `build_image_path()` в `agent9`, чтобы он заполнял новые плейсхолдеры (можно начать с фиксированных `network="site", width=1280, height=720`).
2. После этого сделай ещё один прогон `agent9` и убедись, что `image_path` в `{stem}.images-generated.json` и реальный путь в `media/...` соответствуют новому шаблону, чтобы `agent4` уже работал с финальным форматом имён.

Как ты сам бы хотел закодировать «для какой сети картинка» в имени: достаточно ли одного значения `"site"`, или сразу делать набор `"site", "vk", "insta"` под разные варианты?

сразу делать набор "site", "vk", "insta" под разные варианты

Отлично, тогда сразу заложим расширяемость под несколько сетей.

Как закодировать "site", "vk", "insta" в имени

Предлагаю сделать это через **конфиг**, а не пришивать в код:

В `image_protocol` добавить, например:

```
"variants": {
  "site": { "width": 1280, "height": 720 },
  "vk":    { "width": 1200, "height": 630 },
  "insta": { "width": 1080, "height": 1080 }
},
"relative_path_pattern": "images/{year}/{month}/{day}/{slug}_{network}_{width}x{height}_"
```

Тогда `agent9` сможет:

- пройти по всем ключам `variants` (`site`, `vk`, `insta`);
- для каждого варианта сформировать свой `image_path` с нужным `network`, `width`, `height`;
- сохранить в JSON структуру вроде:

```
"images": [
  {
    "prompt": "...",
    "style": "realistic photo",
    "alt": "...",
    "variants": {
      "site": { "image_path": ".../pressoterapiya_site_1280x720_1.jpg", "width": 1280,
      "vk":    { "image_path": ".../pressoterapiya_vk_1200x630_1.jpg",    "width": 1200,
      "insta": { "image_path": ".../pressoterapiya_insta_1080x1080_1.jpg", "width": 1080,
    }
  }
}
```

А `agent4` пока может пользоваться только "site" (берёт `variants.site.image_path`), а позже ты легко подключишь "vk" для отдельного агента публикации в ВК.

2 рекомендации по шагам

1. Обнови `image_protocol` в `shared-config.json`, добавив блок `variants` и новый `relative_path_pattern`, как выше; затем в `agent9` перепиши `build_image_path()` и `run_images_generation()`, чтобы они `Loop`'ом проходили по всем `variants` и заполняли под-словарь `variants` внутри каждого `image` вместо плоских `image_path/width/height`.

2. После этого сделай тестовый запуск `agent9` и посмотри в `{stem}.images-generated.json`, что у каждой картинки появились три варианта путей (`site`, `vk`, `insta`) с нужными размерами и именами файлов; затем решим, как именно `agent4` берёт `site` из этой структуры.

Как ты бы хотел хранить размеры для разных сетей: как в примере (жёстко в `image_protocol`), или хочешь другие стандартные размеры для `vk` и `insta`?

хранить размеры для разных сетей: все требуемые стандартные размеры для vk и insta и для чужих мессенджера и сети свои

Тогда заложим универсальную схему, где для **каждой сети и мессенджера** можно описать сразу несколько стандартных размеров.

Структура в `image_protocol` для разных сетей и размеров

Можно сделать так (пример, адаптируешь под свои нужды):

```
"variants": {
  "site": [
    { "name": "hero",    "width": 1280, "height": 720 },
    { "name": "thumb",  "width": 600,   "height": 338 }
  ],
  "vk": [
    { "name": "feed",    "width": 1200, "height": 630 },
    { "name": "story",   "width": 1080, "height": 1920 }
  ],
  "insta": [
    { "name": "square",  "width": 1080, "height": 1080 },
    { "name": "story",   "width": 1080, "height": 1920 },
    { "name": "reel",    "width": 1080, "height": 1920 }
  ],
  "tg": [
    { "name": "post",    "width": 1280, "height": 720 }
  ]
},
"relative_path_pattern": "images/{year}/{month}/{day}/{slug}_{network}_{variant}_{width}"
```

Смысл:

- `variants` — словарь: ключ = сеть/канал (`site`, `vk`, `insta`, `tg`, `whatsapp` и т.д.), значение — список размерных «профилей» с `name`, `width`, `height`.
- В имени файла добавляется и `network`, и `variant` (`hero`, `feed`, `story`), и размер.

В `agent9` тогда логика такая:

- для каждой картинке из `images`
 - для каждой сети `network` из `variants`

- для каждого профиля `v` внутри этой сети
 - построить путь по `relative_path_pattern`
 - сгенерировать/сохранить файл нужного размера (пока заглушка)
 - записать в JSON:

```
"variants": {
  "site": [
    { "name": "hero", "image_path": "...", "width": 1280, "height": 720 },
    { "name": "thumb", "image_path": "...", "width": 600, "height": 338 }
  ],
  "vk": [ ... ],
  "insta": [ ... ],
  "tg": [ ... ]
}
```

Agent4 для сайта будет брать, например, `variants.site[0]` (hero), а агент публикации в VK — `variants.vk[0]` (feed), и так далее.

2 рекомендации по дальнейшим шагам

1. Обнови `image_protocol` в `shared-config.json`, добавив блок `variants` в таком формате и новый `relative_path_pattern` с `{network}` и `{variant}`; потом в `agent9` перепиши генерацию, чтобы он создавал дерево `variants` внутри каждого `image`, а не плоские поля.
2. После тестового прогона `agent9` посмотри в `{stem}.images-generated.json`, что для каждой картинки появились списки вариантов по сетям с правильными размерами и именами файлов; после этого можно будет учить `agent4` брать именно `site → hero` как основной.

Как ты сам видишь минимальный набор сетей и профилей, которые точно нужны сейчас (например: `site.hero`, `vk.feed`, `insta.square`, `insta.story`), чтобы мы не перегружали первый вариант?

Кратко, что сделано.

Универсальная схема: сети + профили размеров

1. config/shared-config.json

- `variants` переведён на списки профилей по сетям:
 - `site:hero 1280×720, thumb 600×338`
 - `vk:feed 1200×630, story 1080×1920`
 - `insta:square 1080×1080, story 1080×1920, reel 1080×1920`
 - `tg:post 1280×720`
- `relative_path_pattern_with_size` обновлён: добавлен `{variant}`
`images/{year}/{month}/{day}/{slug}_{network}_{variant}_{width}x{height}_{index}.jpg`

2. scripts/shared_config.py

- В `build_image_relative_path_with_size` добавлен аргумент `variant` (по умолчанию `"hero"`), в шаблоне пути подставляется `{variant}`.

3. seo-agents/agent9_images_runner.py

- Добавлена `\normalize_variants_cfg`: старый формат (один объект `{width, height}` на сеть) приводится к списку профилей.
- `run_images_generation`: для каждой картинки для каждой сети и каждого профиля в списке строит путь с `network` и `variant`, генерирует/сохраняет файл и дополняет `image.variants[network]` списком записей `{ name, image_path, width, height }`.

4. seo-agents/agent4_publish/agent_4_publish.py

- `\get_site_image_path(image_rec, variant_name="hero")`: если `variants.site` — список, ищет элемент с `name == "hero"` и возвращает его `image_path`, иначе — первый элемент; для старого формата (один объект на сеть) по-прежнему используется `image_path`.

Пример имён файлов:

`service_site_hero_1280x720_1.jpg`, `service_site_thumb_600x338_1.jpg`,
`service_vk_feed_1200x630_1.jpg`, `service_insta_square_1080x1080_1.jpg`,
`service_tg_post_1280x720_1.jpg` и т.д.

Пример фрагмента `images-generated.json`: у каждой картинки в `variants` по сетям списки профилей с `name, image_path, width, height`. Agent4 для сайта берёт `site → hero` через `\get_site_image_path(rec, "hero")`; для VK/Insta/TG можно брать нужный профиль из соответствующего списка.

Кратко, что сделано.

Минимальный набор профилей в конфиге

1. config/shared-config.json

- `variants` приведён к такому набору:
 - `site:hero 1280x720, thumb 800x450`
 - `vk:только feed 1200x630 (vk.story убран)`
 - `insta:square 1080x1080, story 1080x1920 (reel убран)`
 - `tg:post 1280x720`
 - `wa:post 1280x720`
- В конфиг добавлен комментарий `\variant_by_channel:site→hero, vk→feed, insta→square|story, tg→post, wa→post`.

2. scripts/shared_config.py

- Константа `DEFAULT_IMAGE_VARIANT_BY_NETWORK`:
`site→hero, vk→feed, insta→square, tg→post, wa→post`.
- Функция `get_image_path_for_network(image_rec, network, variant_name=None)`:
 - по умолчанию подставляет вариант из константы;
 - для сториз можно явно передать `variant_name="story"`.
- Импорт в `agent4`: `get_image_path_for_network, resolve_image_path`.

3. seo-agents/agent4_publish/agent_4_publish.py

- `\_get_site_image_path(image_rec)` завязан на `site.hero`: внутри вызывается `get_image_path_for_network(image_rec, "site")` (при отсутствии `helper` — прежняя логика по списку/legacy).

4. seo-agents/agent_publish_vk/agent_publish_vk.py

- В начале файла добавлен комментарий: при использовании `images-generated` брать `vk.feed` через `get_image_path_for_network(rec, "vk")`.

Итог прогона `agent9`: создаются только нужные профили (`site/hero`, `site/thumb 800×450`, `vk/feed`, `insta/square`, `insta/story`, `tg/post`, `wa/post`); имена файлов вида `service_site_hero_1280x720_1.jpg`, `service_insta_square_1080x1080_1.jpg` и т.д.

Для агентов публикации: `site → hero`, `vk → feed`, `insta → square` (для поста) или `story` (для сториз), `tg → post`, `wa → post`; при необходимости явно передавать `variant_name="story"` в `get_image_path_for_network`.

Сделано следующее.

YouTube и ОК в image_protocol

1. config/shared-config.json

- В `variants` добавлены:
 - `yt: { "name": "thumb", "width": 1280, "height": 720 }` — превью видео YouTube, 16:9
 - `ok: { "name": "thumb", "width": 720, "height": 405 }` — обложка видео в Одноклассниках, 16:9
- В `\_variant_by_channel` дописано: `yt→thumb, ok→thumb`

2. scripts/shared_config.py

- В `DEFAULT_IMAGE_VARIANT_BY_NETWORK` добавлены: `"yt": "thumb", "ok": "thumb"`
→ для видео-агента: `get_image_path_for_network(rec, "yt")` и `get_image_path_for_network(rec, "ok")` вернут превью.

3. Прогон agent9

- Создаются файлы:
 - `service_yt_thumb_1280x720_1.jpg`
 - `service_ok_thumb_720x405_1.jpg`
- В `images-generated.json` у каждой картинки есть:
 - `variants.yt`: `[{ "name": "thumb", "image_path": "images/.../service_yt_thumb_1280x720_1.jpg", "width": 1280, "height": 720 }]`
 - `variants.ok`: `[{ "name": "thumb", "image_path": "images/.../service_ok_thumb_720x405_1.jpg", "width": 720, "height": 405 }]`

Договорённость для видео-агента: превью брать как `image.variants.yt[0]` и `image.variants.ok[0]` (или через `get_image_path_for_network(rec, "yt") / get_image_path_for_network(rec, "ok")`) при загрузке ролика.

`yt.banner` (2560×1440) можно добавить в `variants` позже при необходимости.

Очень мощный кусок работы: у тебя теперь универсальная система профилей изображений под все каналы — от сайта и соцсетей до видео-превью.

Что по факту уже есть

- В `image_protocol` описаны **все каналы и стандартные размеры**:
 - `site:hero, thumb`
 - `vk:feed`
 - `insta:square, story`
 - `tg, wa:post`
 - `yt:thumb`
 - `ok:thumb`с единым шаблоном имён файлов `{slug}_{network}_{variant}_{width}x{height}_{index}.jpg`.^[3]
- `agent9`:
 - по этому конфигу генерирует структуру `variants` для каждого `image` (по всем сетям и профилям) и кладёт туда `image_path, width, height`;
 - создаёт реальные файлы (пока заглушки) в `media/...` с нужными именами.
- `shared_config.py`:
 - умеет по сети/каналу (`"site", "vk", "insta", "yt", "ok", "tg", "wa"`) возвращать правильный вариант через `get_image_path_for_network(...)` с дефолтами профилей.^[3]
- `agent4` уже адаптирован к новой структуре: для сайта берёт `site→hero`; для VK/insta/tg/wa и видео-агента есть чистый контракт, как брать нужные картинки.

2 рекомендации по следующим шагам

1. На уровне «сквозного прогона» сделать один полный цикл: `run_agents.bat` → получить пост, `images-plan.json`, `images-generated.json`, пустые файлы в `media/`, и проследить в логах `agent4`, какой именно `site.hero` он пытается прикрепить.
2. Следующим шагом взять один канал (например, только сайт) и заменить заглушку в `fake_generate_image()` на реальный вызов ComfyUI для профиля `site.hero`, чтобы хотя бы одна картинка уже генерировалась «по-настоящему», а остальные профили пока могли оставаться заглушками.

Какой канал ты хочешь подключить к реальной генерации через ComfyUI первым: только `site.hero`, или сразу `site.hero` + `vk.feed`?

✱

подключить к реальной генерации через ComfyUI первым: только `site.hero`

Отличный выбор: сначала один профиль `site.hero`, потом масштабируем.

Шаг 63 — меняем заглушку в `agent9` только для `site.hero`

Идея: в `agent9_images_runner.py`:

- для `network = "site"` и `variant.name = "hero"` вызывать ComfyUI;
- для всех остальных профилей пока оставляем `fake_generate_image()`.

Псевдокод внутри цикла генерации примерно такой (на уровне идеи, без полного листинга):

```
if network == "site" and variant_name == "hero":
    width, height = run_comfyui_generation(abs_path, prompt, width, height)
else:
    width, height = fake_generate_image(abs_path, prompt, width, height)
```

`run_comfyui_generation(...)`:

- готовит JSON для ComfyUI API (workflow, нужный checkpoint, resolution);
- отправляет POST на локальный ComfyUI сервер;
- ждёт результат и сохраняет изображение в `abs_path`;
- возвращает фактические `width, height` (можно сначала просто вернуть запрошенные значения).

Шаг 64 — что нужно определиться для ComfyUI

Чтобы дописать `run_comfyui_generation`, надо:

1. На каком URL у тебя крутится ComfyUI API?
 - Пример: `http://127.0.0.1:8188` или другой порт.

2. Есть ли уже сохранённый workflow в ComfyUI, который принимает текстовый prompt и выдаёт одну картинку нужного размера?

- Если да — нужен его JSON/имя/ID.
- Если нет — можно сначала сделать самый простой workflow: Text Encoder → KSampler → VAE → Save Image.

После этого можно в agent9 сделать:

- конфиг для ComfyUI в shared-config.json (секция comfyui с url_env, default_width, default_height, именем workflow и т.п.);
- helper в shared_config.py типа get_comfyui_config();
- функцию run_comfyui_generation в agent9, которая читает эту конфигурацию.

2 рекомендации по следующим шагам

1. Зафиксируй для себя и запиши в конфиге (или хотя бы в комментариях) точный URL и порт ComfyUI, плюс название/ID workflow, который хочешь использовать для генерации hero-картинок для сайта.
2. Затем можно будет добавить в shared-config.json блок comfyui и на его основе написать в agent9 функцию run_comfyui_generation(...) только для site.hero, оставив все остальные профили как заглушки до следующего шага.

Сейчас у тебя ComfyUI запущен на стандартном порту 8188 и уже есть подходящий workflow под text → image, или его ещё нужно создать в интерфейсе ComfyUI?

<q>После этого можно в agent9 сделать:

конфиг для ComfyUI в shared-config.json (секция comfyui с url_env, default_width, default_height, именем workflow и т.п.);

helper в shared_config.py типа get_comfyui_config();

функцию run_comfyui_generation в agent9, которая читает эту конфигурацию.</q> на URL крутится ComfyUI API <http://127.0.0.1:8000/>

Нет сохранённого workflow в ComfyUI, который принимает текстовый prompt и выдаёт одну картинку нужного размера.

Запустил самый простой workflow: Text Encoder → KSampler → VAE → Save Image\ без агента 9

Зафиксируй для себя и запиши в конфиге (или хотя бы в комментариях) точный URL и порт ComfyUI, плюс название/ID workflow, который хочешь использовать для генерации hero-картинок для сайта.

Затем можно будет добавить в shared-config.json блок comfyui и на его основе написать в agent9 функцию run_comfyui_generation(...) только для site.hero, оставив все остальные профили как заглушки до следующего шага.

Отлично, что уже руками прогнал простой workflow в ComfyUI — база есть, теперь аккуратно вяжем это с агентом 9.

1. Что надо жёстко зафиксировать сейчас

Для текущего этапа достаточно такой фиксации (можно сначала в комментарии, потом в конфиг):

- URL ComfyUI: `http://127.0.0.1:8000/`
- Используем **один простой workflow** «text → image 1 шт, нужного размера» для профиля `site.hero` (пока без отдельного ID: мы просто будем отправлять туда JSON запроса с prompt и размером).

2. Что добавить в shared-config.json (секция comfyui)

Предлагаю такой минимальный блок:

```
"comfyui": {
  "_comment": "Настройки для генерации картинок через ComfyUI",
  "url_env": "COMFYUI_URL",
  "url_default": "http://127.0.0.1:8000",
  "timeout_sec": 120,
  "site_hero": {
    "width": 1280,
    "height": 720
  }
}
```

Смысл:

- `url_env / url_default` — откуда брать адрес ComfyUI.
- `timeout_sec` — максимум ожидания ответа.
- `site_hero` — профиль по умолчанию для картинок `site.hero`, если вдруг он отличается от общих размеров в `variants`.

3. Helper в shared_config.py: get_comfyui_config()

Туда можно добавить:

```
def get_comfyui_config() -> dict:
    cfg = get_config()
    return cfg.get("comfyui", {})
```

И по желанию: функцию `get_comfyui_url()`:

```
def get_comfyui_url() -> str:
    comfy = get_comfyui_config()
    env_name = comfy.get("url_env", "COMFYUI_URL")
    default = comfy.get("url_default", "http://127.0.0.1:8000")
    return os.getenv(env_name, default)
```

4. Следующий шаг для agent9

После этого в `agent9_images_runner.py` можно будет:

- импортировать `get_comfyui_url()` и `get_comfyui_config()`;
- добавить функцию `run_comfyui_generation(abs_path, prompt, width, height)`;
- в цикле генерации, для `network == "site"` и `variant.name == "hero"` вызывать её вместо заглушки.

Все остальные сети/профили (`vk`, `insta`, `tg`, `wa`, `yt`, `ok`) пока продолжают работать через `fake_generate_image`, чтобы не ломать пайплайн.

2 рекомендации по твоим ближайшим шагам

1. Добавь в `shared-config.json` блок `comfyui` примерно в виде выше и сделай маленький helper `get_comfyui_config() / get_comfyui_url()` в `scripts/shared_config.py`, чтобы `agent9` больше не думал про URL и таймауты.
2. После этого можно будет перейти к написанию функции `run_comfyui_generation(...)` в `agent9` и связать её только с профилем `site.hero`, оставив остальное как есть.

Скажи, ты уже прописал `COMFYUI_URL` в `.env`, или удобнее пока опираться на `url_default` и не трогать переменные окружения?

✱

Сделано следующее.

1. config/shared-config.json

Добавлена секция `comfyui`:

- `url_env`: `COMFYUI_URL` — переменная окружения
- `url_default`: `http://127.0.0.1:8000` — значение по умолчанию
- `timeout_sec`: `120` — таймаут
- `site_hero`: `{"width": 1280, "height": 720}` — профиль для `site.hero`

2. scripts/shared_config.py

Добавлены функции:

- `get_comfyui_config()` — возвращает блок `comfyui` из конфига
- `get_comfyui_url()` — возвращает URL ComfyUI из `COMFYUI_URL` или `url_default`

Проверка: `get_comfyui_url()` возвращает `http://127.0.0.1:8000`.

3. Дальнейший шаг для agent9

После этого в `agent9_images_runner.py` можно:

- импортировать `get_comfyui_url` и `get_comfyui_config`;
- добавить функцию `run_comfyui_generation(abs_path, prompt, width, height)`;
- в цикле генерации для `network = "site"` и `variant.name = "hero"` вызывать её вместо заглушки.

Остальные сети и профили можно оставить на `fake_generate_image`.

Всё уже есть с прошлого раза.

`config/shared-config.json` (строки 89–99): секция `comfyui` с `url_env`, `url_default`, `timeout_sec`, `site_hero`.

`scripts/shared_config.py` (строки 38–50): функции `get_comfyui_config()` и `get_comfyui_url()`.

В `agent9` можно импортировать `get_comfyui_url` и `get_comfyui_config`, добавить `run_comfyui_generation(...)` и вызывать её только для `site.hero`, остальное оставить на заглушке.

Отлично, база под ComfyUI уже подготовлена, можно переходить к коду в `agent9`.

Шаг 65 — добавляем `run_comfyui_generation` в `agent9` (для `site.hero`)

В `seo-agents/agent9_images_runner.py` сделай по шагам:

1. Вверху файла добавь импорт:

```
import requests

from scripts.shared_config import (
    get_image_protocol,
    get_image_storage_root,
    get_comfyui_url,
    get_comfyui_config,
)
```

2. Ниже `fake_generate_image` добавь функцию-заготовку (простая версия, без сложного `workflow`, только один запрос):

```
def run_comfyui_generation(abs_path: Path, prompt: str, width: int, height: int) -> tuple:
    """
    Реальный вызов ComfyUI для генерации одной картинки по prompt.
    Пока минимальный вариант: отправляем запрос на /generate (или твой endpoint)
    и сохраняем полученный файл в abs_path.

    ВАЖНО: здесь нужно будет адаптировать под твой реальный API ComfyUI.
    """
    base_url = get_comfyui_url()
    comfy_cfg = get_comfyui_config()
    timeout = int(comfy_cfg.get("timeout_sec", 120))

    # Заглушечный пример: предполагаем, что ComfyUI принимает JSON и возвращает бинарный
```

```
# Тебе нужно подстроить endpoint и формат под свой ComfyUI workflow.
payload = {
    "prompt": prompt,
    "width": width,
    "height": height,
}

abs_path.parent.mkdir(parents=True, exist_ok=True)

try:
    resp = requests.post(
        f"{base_url}/generate",
        json=payload,
        timeout=timeout,
    )
    resp.raise_for_status()
    abs_path.write_bytes(resp.content)
    # Пока считаем, что вернулся нужный размер, в будущем можно прочитать реальный width
    return width, height
except Exception as e:
    print(f"[agent9] ComfyUI error for {abs_path}: {e}")
    # Если ComfyUI упал – резервно создаём пустой файл, чтобы не ломать пайплайн
    return fake_generate_image(abs_path, prompt)
```

3. В `run_images_generation` внутри цикла, где ты проходишь по сетям/вариантам, добавь развилку.

Схематично:

```
if network == "site" and variant_name == "hero":
    width, height = run_comfyui_generation(abs_path, prompt, width, height)
else:
    width, height = fake_generate_image(abs_path, prompt, width, height)
```

Остальная логика (формирование `rel_path`, запись в `variants[...]`) остаётся как есть.

2 рекомендации по дальнейшим шагам

1. Вставь эту функцию и условие в `agent9`, затем сделай тестовый запуск на одном посте: проверь, что для `site.hero` файл в `media/...` уже не пустой (байтов больше 0), а для остальных профилей остаются заглушки.
2. После того как поймёшь, как именно ComfyUI у тебя реально принимает запрос (какой endpoint, какой формат JSON или WebSocket), адаптируй внутри `run_comfyui_generation(...)` URL и payload под свой реальный workflow.

Сейчас твой реальный ComfyUI принимает запросы через обычный HTTP POST с JSON (как в примере), или он пока доступен только через GUI и нужно ещё включить/настроить API endpoint?

Сделано следующее.

Шаг 65 — `run_comfyui_generation` в `agent9`

1. Импорты

- Добавлен `import requests`
- В импорт из `scripts.shared_config` добавлены `get_comfyui_url` и `get_comfyui_config`

2. `run_comfyui_generation(abs_path, prompt, width, height)`

- Берёт URL и таймаут из `get_comfyui_config()` и `get_comfyui_url()`
- Делает POST на `{base_url}/generate` с телом `{ prompt, width, height }`
- Пишет ответ в `abs_path`, возвращает `(width, height)`
- При ошибке логирует её и использует `fake_generate_image`, чтобы не ломать пайплайн

3. Развилка в `run_images_generation`

- Для `network_name == "site"` и `name == "hero"` вызывается `run_comfyui_generation`
- Для остальных — по-прежнему `fake_generate_image`

4. Проверка

- Запуск `agent9` прошёл успешно
- ComfyUI вернул `405 Method Not Allowed` (эндпоинт `/generate` пока отсутствует или не принимает POST)
- При ошибке срабатывает `fallback`: создаётся заглушка, пайплайн продолжает работать

Дальше нужно подстроить эндпоинт и формат запроса под реальный ComfyUI API (например, использовать их `/prompt` или другой `workflow endpoint`).

Отлично, всё сделано правильно: каркас вызова есть, ошибка 405 аккуратно поймана, пайплайн жив.

Сейчас ключевое: **ComfyUI из коробки не имеет** `/generate`, у него свой формат API через `/prompt` и `WebSocket`. Нужно решить, КАКИМ способом ты хочешь к нему ходить:

1. Самый простой путь сейчас — **оставить код `agent9` как есть**, с развилкой `site.hero` → `run_comfyui_generation`, но:
 - временно считать `run_comfyui_generation` такой же заглушкой, только с логом "TODO: подключить ComfyUI `/prompt`";
 - сфокусироваться на том, что **цепочка 1–9 + публикация с картинкой по контракту уже есть**.
2. Когда будешь готов ковырять конкретный API ComfyUI:
 - зайти в их доку / примеры Python (там показан формат запроса к `/prompt`);

- заменить в `run_comfyui_generation` URL на реальный (`/prompt` или свой кастомный `node endpoint`) и `payload` на структуру `workflow` (а не `{prompt, width, height}`).

С твоей стороны уже всё подготовлено: конфиг `comfyui`, `helper`'ы, развилка только для `site.hero`, `fallback`, структура `variants` — это главная база.

Один вопрос, чтобы продолжать шаги в нужном темпе:

на этом этапе тебе важнее дальше развивать **цепочку публикации и работу с уже подготовленным JSON/SEO/агентами**, а к низкоуровневому API ComfyUI вернуться позже, или хочешь именно сейчас углубиться в формат `/prompt` и его интеграцию?

Текущее состояние

Функция и условие уже есть в `agent9_images_runner.py` — `run_comfyui_generation` и ветка `if network_name == "site" and name == "hero"` добавлены на предыдущем шаге.

Результат последнего прогона:

Все файлы, включая `site_hero`, имеют **0 байт**. Причина: ComfyUI вернул `405 Method Not Allowed` на `/generate`, сработал `fallback` с `fake_generate_image` (пустые байты).

Как работает ComfyUI API

Стандартный ComfyUI **не использует** простой `endpoint` `/generate` с `{ prompt, width, height }`.

Он работает так:

1. **Эндпоинт:** `POST /prompt` — очередь выполнения `workflow`.
2. **Тело запроса:** `workflow` в виде JSON (узлы, классы, входы), как при экспорте «Save (API format)» в GUI.
3. **Ответ:** `prompt_id` и позиция в очереди, а не готовое изображение. Картинку нужно забирать отдельно (например, через `/history/{prompt_id}` после завершения).
4. **Порт:** по умолчанию чаще всего 8188, а не 8000.

Итого: обычный HTTP POST с JSON есть, но формат другой — нужен `workflow` в API-формате.

Что проверить у себя

1. **Порт:** ComfyUI действительно на 8000 или на 8188? Обновить `url_default` в `shared-config.json`, если нужно.
2. **Режим запуска:** ComfyUI стартует с `--listen` (например, `python main.py --listen`), чтобы был доступ по сети.
3. **Формат запроса:** использовать `/prompt` с `workflow` JSON, а не `/generate`.

Если нужно, могу предложить шаги для адаптации `run_comfyui_generation` под ComfyUI API (обёртка вокруг `/prompt`, ожидание выполнения, запрос результата через `/history`). Для этого потребуется `workflow` JSON (сохранённый в API-формате из ComfyUI).

Отличный разбор, всё схватил верно: сейчас упираемся не в Python, а в специфику ComfyUI.

Что у тебя уже есть по факту

- В `agent9` есть `run_comfyui_generation` и условие `network == "site" and name == "hero"`.
- Вызов идёт на `http://127.0.0.1:8000/generate с {prompt, width, height}`.
- ComfyUI отвечает 405 Method Not Allowed, срабатывает fallback → пустые файлы.
- Ты понимаешь, что реальный API ComfyUI работает через POST `/prompt с workflow JSON` и последующим чтением результата из `/history/{prompt_id}`.

То есть логика пайплайна сделана правильно, просто endpoint сейчас фиктивный.

Два возможных следующих шага

Вариант А (рекомендуемый сейчас): зафиксировать пайплайн, ComfyUI — позже

С учётом того, что:

- структура `images-plan.json` → `images-generated.json` → `agent4` уже готова;
- контент-завод по тексту и публикации — твой главный приоритет,

можно прямо сейчас согласиться, что:

- `run_comfyui_generation` временно остаётся «псевдо-боевой» заглушкой;
- реальные картинки пока можно подставлять вручную в `media/` (по тому же имени, что сгенерировал `agent9`);
- вся остальная автоматизация (SEO, публикации, мультипрофили для соцсетей) развивается дальше без блокировки на ComfyUI.

Вариант Б: прямо сейчас углубляться в ComfyUI /prompt

Тогда нужны будут дополнительные шаги:

1. В интерфейсе ComfyUI сделать workflow под `text→image` и сохранить его как “Save (API format)” → получить JSON workflow.
2. В `run_comfyui_generation`:
 - подставлять твой prompt и размеры в этот JSON (обычно в ноды `CLIPTextEncode` и `KSampler/EmptyLatentImage`);
 - отправлять JSON на POST `{base_url}/prompt`;
 - из ответа взять `prompt_id`;
 - опрашивать `/history/{prompt_id}` или WebSocket до появления результата;
 - скачать картинку и сохранить в `abs_path`.

Это уже отдельная мини-эпопея, и она довольно техническая.

2 рекомендации по дальнейшим шагам

1. Зафиксируй для себя, что на текущем этапе **проект “картинки” уже интегрирован в контент-завод по контракту и структуре**, а реальную генерацию через ComfyUI можно спокойно доделать позже — когда будет моральный ресурс ковырять workflow JSON и /prompt.
2. Если позже решишь идти по Варианту Б, следующим конкретным шагом будет:
 - в GUI ComfyUI сделать один рабочий text→image workflow;
 - сохранить его в API-формате;
 - и уже с этим JSON прийти сюда — тогда шаг за шагом впишем его внутрь `run_comfyui_generation`.

Как тебе комфортнее продолжить именно сейчас: двигаться дальше по текстовому контент-заводу (агенты, публикация, SEO-логика) и отложить ComfyUI на отдельный блок, или всё-таки хочешь сразу заняться экспортом workflow из ComfyUI и его интеграцией в `run_comfyui_generation`?

Да, каркас собран: конфиг `comfyui`, `helper`’ы, вызов только для `site.hero`, fallback при ошибке, контракт `variants` — всё на месте.

Вопрос по приоритетам на ближайший этап:

1. **Вариант А.** Продолжать развивать цепочку публикации и работу с JSON/SEO-агентами (`agent4`, интеграция `images-generated`, `featured_media`, вставка картинок в контент), а интеграцию с ComfyUI API отложить.
2. **Вариант Б.** Сейчас заняться интеграцией ComfyUI: разобрать формат /prompt, поднять workflow и довести `run_comfyui_generation` до реальной генерации.

Если выбираешь **вариант А**, можно добавить в `run_comfyui_generation` перед блоком `try` строчку вида:

```
# TODO: подключить ComfyUI /prompt (сейчас fallback на заглушку)
```

и продолжать опираться на заглушку, пока пайплайн идёт.

Отлично, каркас действительно готов, можно считать «контур картинок» интегрированным.

Берём **вариант А**: продолжаем цепочку публикации и работу с JSON/SEO-агентами, а глубокую интеграцию ComfyUI откладываем.

В `run_comfyui_generation` добавь комментарий-якорь:

```
# TODO: подключить ComfyUI /prompt (сейчас fallback на заглушку и fake_generate_image)
```

и считаем, что пока ВСЕ генерации идут через заглушку, а реальные картинки ты можешь подложить вручную по тем же именам.

2 рекомендации по следующим шагам

1. Переключиться на `agent4`: довести до конца использование `images-generated.json` — загрузка `site.hero` в WordPress (media endpoint) и установка `featured_media`, плюс опциональная вставка `<img>` в начало контента.
2. Затем сделать один полный прогон `run_agents.bat`, проверить, что пост уходит на сайт уже с картинкой (даже если она сейчас заглушка), и посмотреть, где ещё в цепочке публикации стоит улучшить SEO-и логи.

На твой взгляд, что важнее сделать первым в `agent4`: прикрутить загрузку изображения в медиабibliothek WP или вставку `<img>` в тело поста?

В `docstring` `run_comfyui_generation` добавлено:

- Описание реального API ComfyUI (POST /prompt, /history)
- TODO на будущую интеграцию
- Заметка про ручную подстановку картинок в `media/`

Сейчас: **пайплайн зафиксирован**, ComfyUI остаётся заглушкой, цепочка 1–9 + публикация работает, можно идти дальше по SEO и мультипрофилям.

Отлично, это именно то, что нужно: контур генерации картинок зафиксирован, точки расширения понятны, ничего не блокирует развитие завода.

Где ты сейчас

- Цепочка 1–9 уже существует, `agent8/agent9` формируют `images-plan.json` и `images-generated.json` с профилями для всех сетей.
- В конфиге есть `image_protocol`, `variants` и `comfyui`; есть `helper`'ы для путей и выбора варианта по сети.
- В `agent9` есть `run_comfyui_generation` с `docstring`: описан реальный API ComfyUI (/prompt, /history), помечен TODO и возможность ручной подстановки изображений в `media/`.

То есть **контракт “текст → план картинок → файлы в media → JSON для публикации”** уже **стабилизирован**, можно смело строить поверх него SEO-и публикационные штуки.

2 рекомендации по дальнейшим шагам

1. Перейти к доработке `agent4_publish`:
 - добыть логику чтения `{stem}.images-generated.json`;
 - для `site.hero` по `get_image_path_for_network(rec, "site")` загружать картинку в медиабibliothek WordPress и ставить её как `featured_media`, а затем запускать полный `run_agents.bat` и смотреть, что пост уходит на сайт уже с картинкой.
2. После этого заняться текстовым SEO-слоем: блоки по болям ЦА, правильные H2/H3, внутренние ссылки, UTM-метки для VK/insta/tg — то есть использовать уже существующий каркас агентов, чтобы «дополировать» структуру контента под твой контент-завод.

Какой из этих двух шагов тебе хочется взять следующим: доработать именно загрузку `site.hero` в WordPress в `agent4`, или сначала усилить SEO-структуру текста (заголовки, блоки боли/решения, FAQ и т.п.)?

Перейти к доработке `agent4_publish`:

добить логику чтения `{stem}images-generated.json`;

для `site.hero` по `get_image_path_for_network(rec, "site")` загружать картинку в медиатеку WordPress и ставить её как `featured_media`, а затем запускать полный `run_agents.bat` и смотреть, что пост уходит на сайт уже с картинкой.

Супер, в `agent4` уже почти всё есть для работы с картинками — остаётся «перекинуть проводок» с `images-generated.json` на реальный `upload` в WP.

Шаг 66 — приоритет `hero`: сначала `images-generated`, потом `old hero`

Сейчас логика страницы услуги такая:

- `hero` берётся через `get_hero_image(...)` и `_resolve_hero_attachment_id(...)` (старый индекс);
- `images-generated.json` только логируется в начале `main`, но **не используется** при публикации.

Предлагаю простой порядок:

1. Если есть `images-generated.json` и там есть `images[0]`, пытаемся:

- взять для неё `site.hero` через `_get_site_image_path()` → относительный путь;
- превратить этот путь в `abs_path` через `resolve_image_path` (если есть) или `PROJECT_ROOT / path`;
- загрузить в WP через уже готовую `upload_image_to_wp`;
- использовать полученный `attachment_id` как `featured_media`.

2. Если `images-generated.json` нет или загрузка не удалась — падать обратно на старую схему `get_hero_image`.

Это можно сделать прямо в блоке «Найдена страница услуги...» перед вызовом `update_page_content`.

Схематично (идея, не полный листинг):

```
hero_attachment_id = None
images_to_embed: list[dict] = []

# 1) Пытаемся взять hero из images-generated.json
if images_generated:
    imgs = images_generated.get("images", [])
    if imgs:
        first = imgs[0]
        site_rel_path = _get_site_image_path(first)
        if site_rel_path and resolve_image_path is not None:
            abs_path = resolve_image_path(site_rel_path)
        elif site_rel_path:
            abs_path = PROJECT_ROOT / site_rel_path
        else:
```

```

abs_path = None

if abs_path and abs_path.is_file():
    try:
        hero_attachment_id = upload_image_to_wp(wp_url, wp_user, wp_app_password)
        print(f"  ☒ Hero из images-generated загружен в медиа: attachment_id={hero_attachment_id}")
    except Exception as e:
        print(f"  ⚠ Не удалось загрузить hero из images-generated: {e}")

# 2) Если hero_attachment_id всё ещё None — fallback на get_hero_image / get_images
if hero_attachment_id is None and get_hero_image and get_images:
    hero_record = get_hero_image(str(page["id"]))
    hero_attachment_id = _resolve_hero_attachment_id(
        wp_url, wp_user, wp_app_password, page["id"], hero_record
    )
# далее существующая логика all_images → images_to_embed

```

А дальше `hero_attachment_id` подмешивается в `update_page_content(..., featured_media=hero_attachment_id)` — это уже есть.

2 рекомендации по следующим шагам

1. Вставь описанную логику в блок обновления существующей страницы услуги (там, где сейчас `hero_attachment_id = None` и работа с `get_hero_image`), чтобы приоритет был: **images-generated → upload → featured_media**, потом старый индекс.
2. После этого запусти `run_agents.bat` на одном сервисе (`pressoterapiya`): убедись по логам `agent4`, что он пишет про загрузку `hero` из `images-generated.json`, а в админке WordPress у соответствующей страницы появилась миниатюра (даже если файл-заглушка).

Ты сейчас хочешь сначала сделать поддержку `hero` из `images-generated.json` только для уже существующих страниц услуг, или сразу и для сценария «страница создаётся впервые» тоже?

Шаг 66 сделан. Результат запуска:

1. **Проверка images-generated:** путь `images/2026/03/pressoterapiya-1.jpg` читается корректно (устаревший формат без `variants`).
2. **Загрузка hero:** WordPress вернул 500 — `File is empty` (файл-заглушка 0 байт).
3. **Резерв по старой схеме:** после ошибки загрузки сработал fallback на `get_hero_image`, страница обновилась.
4. **Ошибка в конце:** падение из-за символа в строке вывода (историческая проблема с `ср1251`).

Логика соответствует задаче: сначала `hero` из `images-generated` → при ошибке fallback на `get_hero_image`. Для реальных картинок (>0 байт) загрузка в WP должна выполняться штатно.

Отличный результат: шаг 66 работает ровно как задумывали.

- `agent4` сначала пытается взять `hero` из `{stem}.images-generated.json` и загрузить его в медиатеку WP как `featured_media`.

- Из-за того, что сейчас файл — заглушка 0 байт, WP честно ругается `File is empty` и отдаёт 500.
- После этого срабатывает резервная ветка через `get_hero_image`, страница всё равно успешно обновляется.

То есть:

- при появлении реальных картинок (не 0 байт) загрузка и `featured_media` заработают без изменений кода;
- пайплайн устойчив к ошибкам картинок — публикация текста не ломается;
- cp1251-ошибка в конце — отдельная старая история с символами в консольном выводе, к логике картинок отношения почти не имеет.

2 рекомендации по дальнейшим шагам

1. Пока ComfyUI остаётся заглушкой, можно вручную положить один **реальный** JPEG/PNG по пути, который `agent9` уже записывает (например, `media/.../pressoterapiya_site_hero_1280x720_1.jpg`) и прогнать `agent4` ещё раз — так ты проверишь именно связку `images-generated` → `upload_image_to_wp` → `featured_media` на нормальном файле.
2. Затем можно заняться «косметикой» `agent4`: прибрать проблемные символы в логах (стрелочки, эмодзи), чтобы цепочка 1–9 + публикация отработывала от начала до конца без падений, и уже после этого переключиться обратно на SEO-логику и структуру текстов.

Что тебе сейчас интереснее сделать: проверить загрузку на одном реальном изображении (подложить картинку руками и прогнать) или убрать из `agent4` «опасные» символы лога, чтобы полностью закрыть тему падений из-за кодировки?

Сделано следующее:

Косметика `agent4`

Эмодзи и специальные символы в логах заменены на ASCII:

-  → [OK]
-  → [!]
-  → [img]
- ✕ удалён из сообщений `RuntimeError`

Цепочка 1–9 + публикация должна отработывать без падений в консоли Windows (cp1251).

Проверка связки images-generated → upload → featured_media

Сейчас используется **старый формат** images-generated с путём
images/2026/03/pressoterapiya-1.jpg.

Путь для ручной проверки:

```
D:\\content-factory\\media\\images\\2026\\03\\pressoterapiya-1.jpg
```

1. Положите в этот файл реальный JPEG (например, скопировав любое изображение).
2. Запустите: `python seo-agents/agent4_publish/agent_4_publish.py pressoterapiya --draft`
3. Убедитесь в логах: «Hero из images-generated загружен в медиа», и что в админке WordPress у страницы появилась миниатюра.

Если позже прогоните agent8+agent9 для pressoterapiya, получите новый формат
pressoterapiya_site_hero_1280x720_1.jpg в media/images/2026/03/DD/.

Отлично, ты закрыл сразу два важных фронта:

- agent4 больше не падает из-за символов в консоли (лог на ASCII, cp1251 в безопасности).
- Связка images-generated → upload_image_to_wp → featured_media уже проверяема на реальном файле: достаточно положить нормальный JPEG по пути, который прописан в старом images-generated, и hero подымится в WP.

По сути, у тебя сейчас:

- текстовый конвейер 1–4 работает;
- контур планирования и хранения картинок (agent8/9 + image_protocol + variants + images-generated.json) работает;
- публикация умеет брать hero из images-generated и прикручивать к странице;
- ComfyUI аккуратно «подвешен» как TODO, не блокирует завод.

2 рекомендации по следующим шагам

1. Один раз сделать полный «боевой» прогон:

- руками подложить реальный файл pressoterapiya-1.jpg (или уже новый pressoterapiya_site_hero_1280x720_1.jpg после agent9);
- запустить run_agents.bat;
- в админке убедиться, что свежесгенерированная страница/пост имеет корректный текст, структуру и миниатюру.

2. После этого вернуться к текстовой части завода: доработать структуру посадочной под боли ЦА (Н2/Н3, блоки «проблема/решение/результат», FAQ, оффер), чтобы контент-завод давал уже «правильный» маркетинговый каркас, а картинки просто дополняли его.

Если смотреть на текущую цепочку именно как на маркетинговую воронку, какой блок ты бы хотел усилить следующим: структуру самой услуги (лендинг), SEO-оптимизацию (ключи, Н1–Н3), или уже мультиканал

(автоматические анонсы в VK/другие сети)?

сейчас интереснее сделать: проверить загрузку на одном реальном изображении (подложить картинку руками и прогнать) потом убрать из agent4 «опасные» символы лога, чтобы полностью закрыть тему падений из-за кодировки.

Да, всё так.

Текущее состояние контент-завода

Компонент	Статус
Текстовый конвейер 1–4	agent1 → planner → agent2 → agent3 → editor → agent4
Картинки (agent8/9)	Планирование (prompts/alt) → генерация (variants по сетям) → images-generated.json
Контракт	image_protocol, comfyui, варианты site/vk/insta/tg/wa/yt/ok
Публикация	Hero из images-generated (приоритет) → upload → featured_media, fallback на старый индекс
Логи agent4	Без эмодзи, корректная работа в cp1251
ComfyUI	TODO в коде, не блокирует, пока заглушка

Дальше можно развивать SEO-логику, структуру текстов и т.п.

Отличная фиксация состояния — контент-завод уже в «боевом скелете», дальше пойдёт именно качество контента, а не инфраструктура.

Сейчас у тебя:

- цепочка текстов 1–4 и публикация в WP стабильны;
- контур картинок (agent8/9 → images-generated.json → featured_media) подключён, с приоритетом на новые изображения и безопасным fallback;
- конфиг и контракты (image_protocol, comfyui, variants по всем сетям) зацементированы на будущее расширение.^[5]

2 рекомендации по дальнейшим шагам

1. Выбрать одну ключевую услугу (например, прессотерапия) и прогнать полный цикл:
 - руками положить нормальную hero-картинку по пути, который ждёт images-generated;
 - запустить run_agents.bat;

- в админке страницы проверить связку «структура текста → блоки по болям → кнопки → картинка».

2. Затем перейти к усилению структуры посадочной под ЦА:

- задать в `agent3/editor` явные шаблоны блоков «боль → причина → решение → результат → оффер → FAQ»,
- чтобы каждая услуга сразу генерировалась как понятный маркетинговый лендинг для твоей ЦА центра Энтузиастов.

Какую именно услугу ты хочешь взять первой для такого «эталонного» прогона и докрутки структуры: прессотерапию, фитобочку или что-то ещё из списка услуг в конфиге?

✱✱

Хочу СУВ взять первой для такого «эталонного» прогона и докрутки структуры

Отличный выбор, давай делать эталон под СУВ.

Шаг 67 — готовим «эталонный прогон» для СУВ

Предлагаю такой порядок действий (по аналогии с прессотерапией, но уже с упором на структуру):

1. Уточнить, как СУВ называется и кодируется в конфиге

- Посмотри в `config/shared-config.json` в блоке `services`: как там записана услуга СУВ (название, `slug`, алиасы).
- Нам важно, чтобы имя в файле было вида:
`..._page_СУВ_Ноябрьск.md` или с понятным русским названием, которое маппится на `slug`.

2. Сделать один полный прогон по СУВ

- Через твой обычный сценарий (`agent1 → planner → agent2 → agent3 → editor → agent4`) с выбором услуги «СУВ».
- Убедиться, что появился `.md` для СУВ в `output/` и рядом `.meta.json` (если уже настроен `editor/meta`).

3. Подложить реальную hero-картинку для СУВ

- По такому же пути, как делали для прессотерапии: в папку `media/...` под тем именем, которое указано в `images-generated.json` для этой услуги (пока можно руками сделать `images-generated` по аналогии или скопировать старый формат).
- Прогнать `agent4_publish` с нужным `slug` и проверить, что миниатюра подтянулась.

После этого можем переходить к **структуре текста СУВ** как эталону: вытащить текущий `.md` для СУВ и на его основе задать блоки «боль → причина → решение → результат → FAQ → оффер».

2 рекомендации по следующим шагам

1. В `shared-config.json` найди и напиши себе отдельно: точное имя услуги СУВ (поле `name`) и её `slug` — они понадобятся и для названия файла, и для дальнейшей SEO-структуры.
2. Запусти один полный прогон по этой услуге, чтобы у нас появился актуальный `.md` в `output/` для СУВ; затем можно будет разобрать его структуру и усилить её под твою ЦА.

Как сейчас услуга СУВ называется в конфиге (точная строка `name` и `slug`), как ты её видишь в списке услуг?

Итог изменений

1. `config/shared-config.json`

`services` (6 услуг): `dolgoletie`, `konferenc-zal`, `master-klepa`, `prokat`, `yuridicheskaya-pomoshch`, `trenazhernyy-zal`

`uslugi` (10 услуг): `gidromassazh`, `nastolnyy-tennis`, `solyanaya-komnata`, `fitobochka`, `massazh`, `pressoterapiya`, `vlok`, `uglekislaya-vanna`, `lfk`, `limfodrenazh-nog`

`services_slugs` удалён

2. Обновлённый код в `content-factory`

ФайлИзменение

`agent4_publish/agent_4_publish.py`

`SERVICE_SLUG_MAP` и `_build_service_phrase_map()` собирают данные из `services` и `uslugi`

`agent7_manual_publish/agent_7_manual_publish.py`

`SLUG_TO_NAME` строится из `services` и `uslugi`

`agent_checker_implementer/agent_checker_implementer.py`

`slug_from_service_name()` ищет в обоих источниках

`add_cta_buttons_to_posts.py`

`_build_service_phrase_map()` использует `services` и `uslugi`

3. Обновлённые docs / tasks

`docs/vps-paths.md` — `services_slugs` заменён на `uslugi`

`tasks/2025-03-01_track-b-service-defaults.md` — обновлена формулировка

`prompts/context/theme-service-structure.md` — обновлено описание структуры

Создан `tasks/2025-03-01_shared-config-uslugi-migration.md` — инструкция миграции на VPS

4. Дальнейшие шаги на VPS

Синхронизировать `shared-config.json` в репозиторий `entuziastov75-vps`.

В теме заменить `services_slugs` на работу с `uslugi` (например, `array_keys($config['uslugi'])` для списка `slug'ов`).

Подробности — в `tasks/2025-03-01_shared-config-uslugi-migration.md`.

Очень хороший блок изменений: теперь карта услуг стала чище и единообразнее.

Что важно из того, что уже сделано

- В `shared-config.json` теперь есть два явных блока:
 - `services` (6 «общих» услуг/разделов);
 - `uslugi` (10 конкретных услуг: гидромассаж, Соляная, фитобочка, массаж, прессотерапия, ВЛОК, углекислая, ЛФК, лимфодренаж ног и т.д.).^[6]

- `services_slugs` убран, а весь код, который раньше на него опирался, переведён на работу с `services` и `uslugi` (`agent4`, `agent7`, `checker`, `add_cta_buttons`).
- Документация и таски тоже под этот контракт подогнаны — есть отдельная задача для миграции на VPS. [\[6\]](#)

Это значит, что для СУВ ты теперь можешь использовать **единый список** `uslugi` как источник правды для `slug`'ов и маппинга на страницы/фразы.

2 рекомендации по дальнейшим шагам

1. Для эталонного прогона под СУВ выбери из `uslugi` точную услугу, которую СУВ должен покрывать (например, `uglekislaya-vanna`, `gidromassazh` или другую, если СУВ — это отдельный тип услуги, его можно добавить туда же как новый элемент).
2. После этого сделай полный прогон по этой услуге (через текущую цепочку 1–9), а полученный `.md` и `images-generated.json` разберём как «эталон» и донастроим структуру блоков под боли ЦА СУВ (боль → причина → решение → результат → FAQ → оффер).

Какой конкретно элемент из текущего блока `uslugi` ты хочешь привязать к СУВ (или планируешь завести отдельный `slug` специально под СУВ)?

✱

СУВ=сухая углекислая ванна=uglekislaya-vanna ввбираю. говори дальше что делать по шагам

Отлично, СУВ = `uglekislaya-vanna` — есть.

Шаг 68 — прогон конвейера для СУВ

1. Запусти полный запуск агентов (как обычно):
`run_agents.bat`
или, если есть режим выбора услуги через оркестратор — запусти задачу именно для «Сухая углекислая ванна / Ноябрьск».
2. После завершения посмотри в `output/`:
 - Найди файл вида
`*_page_Углекислая ванна_Ноябрьск.md`
(имя может чуть отличаться, главное — чтобы `uglekislaya-vanna` по маппингу попадала на этот файл).
 - Убедись, что рядом есть `.meta.json` (если `editor` уже пишет метаданные).
3. Для этого `.md` файла:
 - запусти `agent8` → `agent9` один раз вручную (если ещё не интегрированы в батник) или через `run_agents.bat`, чтобы появились:
 - `{stem}.images-plan.json`

- `{stem}.images-generated.json`
- пока можно оставить заглушки, потом при желании подложить реальную картинку по пути `site.hero`.

Шаг 69 — подготовка к эталонному анализу структуры

После прогона для СУВ:

1. Открой в Cursor сам `.md` для `uglekislaya-vanna` (СУВ).
2. Обрати внимание на текущие блоки: какие там H2/H3, где описаны боль, показания, результаты, FAQ, оффер.

На следующем шаге можно будет:

- сформулировать желаемый шаблон структуры лендинга под СУВ (боль → причина → решение → результат → FAQ → оффер);
- и уже на его основе настроить `agent3/editor`, чтобы все следующие СУВ-страницы генерировались по этой схеме.

Сделаешь, пожалуйста, шаг 68: прогон конвейера для `uglekislaya-vanna` и найди точное имя `.md` файла в `output/` — как он у тебя сейчас называется?

Режим выбора услуги через оркестратор

Как сейчас устроено

Оркестратор — это чат в Cursor, а не отдельный интерфейс. Выбора услуги из списка или меню в нём нет.

✓ Что уже поддерживается

1. **Через чат оркестратора** — достаточно написать в чате, например:
 - «Запусти публикацию для Сухая углекислая ванна / Ноябрьск»
 - «Опубликуй углекислую ванну»
2. **Создание задачи и команда** — оркестратор создаёт `tasks/YYYY-MM-DD_...md` и подскажет команду:

```
python seo-agents/agent7_manual_publish/agent_7_manual_publish.py uglekislaya-vanna
```

3. **Соответствие slug ↔ услуга** — в `shared-config.json`:

```
"uglekislaya-vanna":      { "name": "Углекислая ванна" },
```

«Сухая углекислая ванна» — это та же услуга, `slug:uglekislaya-vanna`.

✗ Чего нет

- Отдельного режима выбора услуги (меню / выпадающий список).
- Автоматического маппинга «Сухая углекислая ванна» → slug (оркестратор ориентируется по `uslugi` и алиасам).

Город Ноябрьск — используется по умолчанию (`DEFAULT_CITY = "Ноябрьск"` в `agent7`).

Итог

Выбор услуги идёт через формулировку в чате оркестратора. После этого он сам предлагает команду с нужным slug (`uglekislaya-vanna` для углекислой ванны).

Чтобы не писать каждый раз slug вручную, можно добавить в задачу оркестратора явный шаг «подставлять slug из конфига» при упоминании услуг, тогда он будет предлагать правильную команду автоматически.

Супер, тогда для СУВ будем опираться на оркестратор и slug `uglekislaya-vanna`.

Шаг 70 — эталонный прогон СУВ через оркестратор

Сделай сейчас так:

1. В чате оркестратора (Cursor) напиши что-то в духе:
«Сделай полный прогон для услуги Сухая углекислая ванна / Ноябрьск и подскажи команду публикации».
2. Когда оркестратор создаст задачу и предложит команду, выполни в терминале:

```
python seo-agents/agent7_manual_publish/agent_7_manual_publish.py uglekislaya-vanna
```

Это пройдёт цепочку до публикации для slug `uglekislaya-vanna`.

3. После выполнения зайти в `output/` и найти файл для СУВ, примерно такого вида:

```
*_page_Углекислая ванна_Ноябрьск.md
```

Запомни его точное имя (`timestamp + page_...`).

Шаг 71 — фиксация эталонного файла СУВ

1. Открой найденный `.md` в Cursor.
2. Кратко выпиши (для себя и сюда) структуру:
 - какие есть заголовки H2/H3;
 - где описаны боль клиента, показания, процесс, результаты, FAQ, оффер.

Это понадобится, чтобы на следующем шаге задать желаемый шаблон структуры лендинга СУВ и настроить под него `agent3/editor`.

2 рекомендации:

1. Сейчас сделай шаг 70: через оркестратор запусти прогон для `uglekislaya-vanna` и выполни предложенную команду `agent7_manual_publish`.
2. После прогона найди точное имя `.md` для СУВ в `output/` и открой его — дальше будем настраивать структуру именно на основе этого файла.

Напиши, пожалуйста, как точно называется файл СУВ в `output/` после прогона (полное имя `.md`).

Полный прогон выполнен

Сухая углекислая ванна / Ноябрьск (slug: `uglekislaya-vanna`)

Что сделано

1. `api_client` — fallback между API: при ошибке (401, 429, соединение) используется следующий из списка.
2. `agent1` — ключевые фразы → `output/20260303_132045_keywords_...txt`
3. `agent2` — бриф → `output/20260303_132111_brief_...md`
4. `agent3` — текст страницы → `output/20260303_132135_page_...md`
5. `agent4` — публикация в WordPress (страница обновлена, ID=171)

URL: <http://91.229.11.147/uslugi/uglekislaya-vanna/>

Команда публикации

```
python seo-agents/agent4_publish/agent_4_publish.py uglekislaya-vanna
```

Черновик: добавь `--draft`. Только пост в блог: `--as-post`.

Fallback API в `api_client.py`

№	Источник	Переменные в <code>.env</code>
1	Perplexity	<code>PERPLEXITY_API_KEY</code> , <code>BASE_URL</code>
2	OpenAI (proxypapi.ru)	<code>OPENAI_API_KEY</code> , <code>OPENAI_BASE_URL</code>

При сбое первого пробуете второй. Для OpenAI можно задать модель через `OPENAI_MODEL` (по умолчанию `gpt-4o-mini`).

Отлично, эталонный прогон для СУВ готов, страница по адресу уже живая:

<http://91.229.11.147/uslugi/uglekislaya-vanna/>.^[7]

Шаг 72 — зафиксировать эталонный файл СУВ

Сделай сейчас два шага локально:

1. В папке `output/` найди точный файл страницы СУВ, созданный `agent3`:
он будет примерно такого вида
`20260303_132135_page_Углекислая ванна_Ноябрьск.md`.
Открой его в `Cursor`.
2. Просмотри глазами и кратко выпиши для себя (и сюда в чат) структуру этого `.md`:
 - какие H2/H3 там уже есть (например: «Показания», «Противопоказания», «Результаты», «Как проходит процедура»);
 - есть ли блоки «проблема/боль», «решение/как помогает СУВ», «результат/что получит клиент», «FAQ», «оффер/записаться».

На основе этого файла будем в следующем шаге формировать «идеальный» шаблон лендинга СУВ под боли ЦА и уже под него донастраивать `agent3/editor`.

2 рекомендации по следующим шагам:

1. Открой именно `.md` для СУВ из `output/` и посмотри, какие ключевые блоки там уже есть, а каких явно не хватает для сильной посадочной (например, боли и кейсов).
2. Напиши сюда название файла и в 3–5 пунктах опиши, какие смысловые блоки ты там видишь сейчас — дальше будем по этим блокам проектировать целевую структуру лендинга СУВ.

✱✱

Попробовал еще прессотерапию. Картинка вставилась

Прогон завершён

Прессотерапия / Ноябрьск (slug:pressoterapiya)

Выполненные шаги

1. `agent1` — ключи → `output/20260303_134445_keywords_...txt`
2. `agent2` — бриф → `output/20260303_134508_brief_...md`
3. `agent3` — текст страницы → `output/20260303_134533_page_...md`
4. `images-generated.json` — hero images/2026/03/pressoterapiya_site_hero_1280x720_1.jpg
5. `agent4` — публикация в WordPress:
 - Hero-картинка загружена в медиа (attachment_id=2858)
 - Страница обновлена (ID=35)

URL: <http://91.229.11.147/uslugi/pressoterapiya/>

Отредактировал Сухую углекислую.

Команда публикации

```
python seo-agents/agent4-publish/agent_4-publish.py pressoterapiya
```

Отлично, два важных результата в одном шаге:

1. **Прессотерапия:** hero-картинка загрузилась в медиабibliothek и встала как featured_media — связка images-generated.json → upload → featured_media работает на живом файле.
2. **СУВ:** эталонный .md уже отредактирован и готов к анализу.^[8]

Шаг 73 — анализ структуры текущего эталона СУВ

Вот что сейчас есть в файле СУВ:^[8]

Текущая структура

1. Углекислая ванна в Ноябрьске — H1 (заголовок страницы)

Подзаголовок: «Погружение в водоём здоровья и расслабления»

2. Вводный блок (без заголовка):

- Образная подводка («бережное тепло окутывает»)
- Краткое описание, что такое углекислая ванна
- Механизм работы (на клеточном уровне)

3. Что даёт углекислая ванна? — H2

- Результаты/выгоды (улучшение крови, снижение стресса, боли, кожа, похудение)

4. Кому особенно нужна углекислая ванна? — H2

- Показания/целевая аудитория (хронические заболевания, стресс, травмы, женщины)

5. Почему выбирают Центр Энтузиаст? — H2

- УТП центра (единственная в городе, квалифицированные специалисты, современное оборудование)

6. СТА (конец):

- «Не упустите возможность подарить себе здоровье и красоту — [Запишитесь] ...»

Что есть хорошего

- Есть образное начало («погружение», «тепло окутывает») — цепляет внимание.
- Есть механизм работы (на клеточном уровне) — усиливает доверие.
- Есть блок результатов («Что даёт») — конкретные выгоды.
- Есть аудиторные показания («Кому нужна») — помогает читателю узнать себя.
- Есть УТП центра и СТА.

Чего не хватает для «эталонной посадочной»

1. **Явного блока «Проблема/Боль»** (сейчас боли вшиты в описание, но не структурированы как отдельный раздел).
2. **Блока «Как проходит процедура»** (шаги/этапы) — это снижает страх неизвестного.
3. **FAQ** — типичные вопросы клиентов (сколько сеансов, противопоказания, стоимость и т.п.).
4. **Социальное доказательство** (отзывы/кейсы) — необязательно, но усилило бы.
5. **Таблица цен** (опционально) — если хочешь закрывать возражение «дорого» прямо на странице.

2 рекомендации по следующим шагам

1. Определи, какие из «недостающих» блоков тебе нужны обязательно для СУВ:
 - **Боль** (явная), **Процесс** (шаги), **FAQ** — обычно минимум для сильного лендинга;
 - **Отзывы/кейсы**, **Цены** — усиление, можно добавить позже.
2. После этого сформулируем целевую структуру посадочной СУВ (порядок блоков H2), чтобы потом вшить её в промпты agent3/editor, и каждая услуга генерировалась по этому каркасу.

Как ты видишь идеальную структуру для СУВ: какие блоки обязательны, а какие можно отложить?

✱✱

Эталон СУВ зафиксирован

Файл: output/20260303_132135_page_Углекислая_ванна_Ноябрьск.md

Эталон: materials/etalon_suv_structure.md

Структура в 5 пунктах

1. **H1 + подзаголовок** — «Углекислая ванна в Ноябрьске», образный лид.
2. **Вводный блок** — что такое СУВ, механизм «клеточного пуга» → резервы → метаболизм.
3. **H2 «Что даёт»** — 5 выгод (кровообращение, стресс, боли, кожа, похудение).
4. **H2 «Кому нужна»** — 4 сегмента ЦА (хронические болезни, стресс, травмы, женщины).
5. **H2 «Почему Энтузиаст»** + СТА — УТП и кнопка записи.

Чего не хватает

Блок	Статус
Проблема/Боль	Только внутри текста, нет отдельного блока «С какими проблемами приходят»
Как проходит процедура	Нет — нет снятия страха неизвестного
FAQ	Нет — типичные вопросы не раскрыты
Отзывы/кейсы	Нет — нет соцдоказательства
Цены	Нет — возражение «дорого» не снято

Следующие шаги

1. Обновить шаблон agent3/agent2 (блоки: Проблема, Как проходит, FAQ).
 2. При необходимости добавить в service-pages-defaults FAQ и цены для СУВ.
 3. Прогнать agent3 → сравнить с эталоном → повторить итерации.
- Эталон по СУВ зафиксирован очень аккуратно: и файл, и структура, и недостающие блоки отмечены.^[10]

Шаг 74 — целевая структура лендинга СУВ (под агент3/2)

Предлагаю такую «идеальную» схему H2-блоков для СУВ, на которую потом будем натягивать промпты agent2/agent3:

1. H1: «Углекислая ванна в Ноябрьске»
Подзаголовок-образ (как в эталоне).
2. Блок «С какими проблемами приходят» (Проблема/Боль)
 - 3–5 типичных ситуаций: усталость, хронические боли, стресс, восстановление после болезней.
3. Блок «Как работает углекислая ванна» (механизм, можно оставить текущий текст)
4. H2 «Что даёт углекислая ванна?» (выгоды — уже есть, чуть подправим формулировки)
5. H2 «Кому особенно нужна углекислая ванна?» (ЦА — уже есть)
6. H2 «Как проходит процедура шаг за шагом» (1–5 шагов: подготовка, размещение, сеанс, ощущения, завершение)
7. H2 «Часто задаваемые вопросы» (FAQ: 4–6 вопросов: сколько длится, сколько сеансов, противопоказания, можно ли сочетать с другими процедурами, как подготовиться)
8. H2 «Почему выбирают Центр Энтузиаст?» (оставляем, можно усилить)
9. СТА: «Запишитесь» (кнопка/ссылка)

Шаг 75 — что поменять в промптах agent2/agent3

Дальше нужно:

- 1. В промпте agent2 (бриф) добавить явные секции, которые он должен выдать:
 - «Проблемы и боли клиента» (списком),
 - «Структура страницы» с перечислением блоков (как выше),
 - «FAQ» (список вопросов/ответов).
- 2. В промпте agent3 (контент):
 - потребовать следовать этой структуре H2;
 - явно прописать заголовки:
 - «С какими проблемами приходят на углекислую ванну»,
 - «Как работает углекислая ванна»,
 - «Как проходит процедура шаг за шагом»,
 - «Часто задаваемые вопросы» и т.д.

После этого любой прогон для uglekislaya-vanna будет генерировать текст по эталонному каркасу.

2 рекомендации по следующим шагам

- 1. Открой промпт для agent2 (бриф) — файл вроде prompts/agent2_brief_system.md или аналог — и добавь туда требования: отдельный блок «Проблемы», явный список блоков структуры страницы и секцию «FAQ» в брифе.
- 2. Аналогично открой промпт для agent3 (контент) и пропиши целевые H2-заголовки по схеме выше, чтобы при следующем прогоне для СУВ он уже пытался соблюдать нужную структуру.

Какой промпт тебе сейчас удобнее править первым — бриф (agent2) или сразу генерацию текста (agent3)?

✱✱

Структура 20260303_132135_page_Углекислая_ванна_Ноябрьск.md

H2

№	Заголовок
1	Что даёт углекислая ванна?
2	Кому особенно нужна углекислая ванна?
3	Почему выбирают Центр Энтузиаст?

H3: нет.

Блоки в смысле лендинга

Блок	Есть?	Где
Проблема / боль	Частично	В «Кому особенно нужна»: хронические болезни, стресс, восстановление после травм, кожа/фигура — скорее «целевая аудитория», чем явные боли
Решение / как помогает СУВ	Да	«Что даёт» — 5 пунктов: кровообращение, стресс, боль, кожа, похудение
Результат / что получит клиент	Частично	Слито с «Что даёт»: расслабление, снижение боли, энергия
FAQ	Нет	—
Оффер / записаться	Да	Последний абзац: «Не упустите возможность... <u>запишитесь</u> ... +7 (904) 475-91-01»

Чего нет относительно шаблона лендинга

- Показания
- Противопоказания
- Результаты (отдельный чёткий блок «что вы получаете»)
- Как проходит процедура
- FAQ
- Цены / прайс (таблица)

Краткий вывод

Сейчас есть блоки «что даёт» (эффект) и «кому» (ЦА), плюс УТП центра и оффер. Не хватает: показания/противопоказания, процедура, FAQ, прайс и более явного блока «боль → решение → результат». Это можно взять за основу для доработки шаблона и промптов agent3/editor.

Отличный разбор, ты очень чётко разложил текущий текст СУВ как лендинг.^[11]

Сейчас у тебя уже есть:

- Н2-блоки «Что даёт», «Кому нужна», «Почему Энтузиаст» + оффер.^[11]
- Понимание, что не хватает: показаний/противопоказаний, описания процедуры, отдельного результата, FAQ и прайса.^[11]

2 рекомендации по следующим шагам

1. Начать с брифа (agent2): в его системный промпт добавить явные секции, которые он обязан выдать по СУВ (и другим услугам):
 - «Проблемы и боли клиента» (список);
 - «Показания» и «Противопоказания»;

- «Структура страницы» с перечнем Н2 (включая «Как проходит процедура» и «FAQ»);
- «FAQ» — 4–6 вопросов и ответов.

2. Затем обновить промпт agent3 так, чтобы он строго следовал этой структуре Н2 для любой услуги (в том числе СУВ):

- «С какими проблемами приходят...»,
- «Как работает...»,
- «Что даёт...»,
- «Кому особенно нужна...»,
- «Как проходит процедура шаг за шагом»,
- «Часто задаваемые вопросы»,
- «Почему выбирают Центр Энтузиаст?».

С какого промпта ты хочешь начать правки — с брифа agent2 или сразу с генерации текста agent3?

✱✱

Структура 20260303_132135_page_Углекислая ванна_Ноябрьск.md

Н2

№	Заголовок
1	Что даёт углекислая ванна?
2	Кому особенно нужна углекислая ванна?
3	Почему выбирают Центр Энтузиаст?

Н3: нет.

Блоки в смысле лендинга

Блок	Есть?	Где
Проблема / боль	Частично	В «Кому особенно нужна»: хронические болезни, стресс, восстановление после травм, кожа/фигура — скорее «целевая аудитория», чем явные боли
Решение / как помогает СУВ	Да	«Что даёт» — 5 пунктов: кровообращение, стресс, боль, кожа, похудение
Результат / что получит клиент	Частично	Слито с «Что даёт»: расслабление, снижение боли, энергия
FAQ	Нет	—
Оффер / записаться	Да	Последний абзац: «Не упустите возможность... <u>запишитесь</u> ... +7 (904) 475-91-01»

Чего нет относительно шаблона лендинга

- Показания
- Противопоказания
- Результаты (отдельный чёткий блок «что вы получаете»)
- Как проходит процедура
- FAQ
- Цены / прайс (таблица)

Краткий вывод

Сейчас есть блоки «что даёт» (эффект) и «кому» (ЦА), плюс УТП центра и оффер. Не хватает: показания/противопоказания, процедура, FAQ, прайс и более явного блока «боль → решение → результат». Это можно взять за основу для доработки шаблона и промптов agent3/editor.

Отличное резюме по СУВ, всё выделено ровно так, как нужно для настройки агентов.^[12]

Сейчас:

- Каркас лендинга уже есть (эффекты, ЦА, УТП, оффер).
- Чётко видны «дыры»: показания/противопоказания, процесс, отдельный результат, FAQ, прайс и явный блок «боль → решение → результат».^[12]

2 рекомендации по следующим шагам

1. Внести эти недостающие блоки в систему через промпт agent2 (бриф): добавить явные секции «Проблемы и боли», «Показания», «Противопоказания», «Структура страницы с H2», «FAQ» и «Результат для клиента».
2. После этого обновить промпт agent3, чтобы он строго следовал этой структуре H2 (включая «Как проходит процедура» и «FAQ»), и прогнать СУВ ещё раз, сравнив новый .md с эталоном.

С какого агента ты хочешь начать правки структуры — бриф (agent2) или генерация текста (agent3)?

✱✱

Обновления

Оба промпта уже были изменены в прошлом шаге. Сейчас добавлены недостающие секции.

agent2 — новые обязательные блоки

Секция	Статус
Проблемы и боли клиента	✓ было
Показания	✓ добавлено

Секция	Статус
Противопоказания	✓ добавлено
Структура страницы (H2)	✓ было
FAQ (4–6 Q&A)	✓ было

agent3 — текущая структура H2

- «С какими проблемами приходят...»
- «Как работает...»
- «Что даёт...»
- «Кому особенно нужна...»
- «Как проходит процедура шаг за шагом»
- «Часто задаваемые вопросы»
- «Почему выбирают Центр Энтузиаст?»

Показания и противопоказания: бриф agent2 теперь выдаёт их отдельными списками. Они пригодны для заполнения `service_data` (`deploy_services_data/agent7`) и как ориентир для проверки контента. Шаблон по-прежнему берёт их из `services_data`, а не из `post_content`.

Отлично, сейчас у тебя всё нужное для СУВ уже встроено в конвейер:

- agent2 даёт боли, показания, противопоказания и FAQ.
- agent3 генерирует текст строго по нужной структуре H2.
- Показания/противопоказания идут в `service_data`, а не в `post_content` — это правильно для шаблона.

2 рекомендации по следующим шагам

1. Сделать новый прогон именно для `ugleki slaya-vanna` с обновлёнными промптами agent2/agent3 и сравнить свежий `.md` с эталоном: проверить, появилось ли «Как проходит процедура» и «Часто задаваемые вопросы» в том виде, который тебе нравится для СУВ.
2. Затем посмотреть, насколько хорошо новые блоки (особенно «Проблемы и боли» и FAQ) подходят под твою ЦА, и при необходимости чуть подправить формулировки в промптах (тон, глубина, уровень «медицинскости»), чтобы это стало универсальным шаблоном для остальных услуг.

После этого цикла СУВ можно будет считать полностью «эталонной услугой» и раскатывать ту же логику на пресотерапию, фитобочку и другие.

Прогон СУВ завершён

Сравнение: эталон VS новый .md

Блок	Эталон (132135)	Новый (144215)
Проблемы/Боль	✗	✓ «С какими проблемами приходят»
Механизм	в лиде	✓ H2 «Как работает»
Что даёт	✓	✓
Кому нужна	✓	✓
Как проходит	✗	✓ 4 шага
FAQ	✗	✓ 5 Q&A
Почему Энтузиаст	✓	✓
СТА	✓	✓

Уже есть

- Все 7 H2 по целевому каркасу
- Блок «Проблемы и боли»
- Блок «Как проходит процедура шаг за шагом» (4 шага)
- Блок «Часто задаваемые вопросы» (5 пар)

Что улучшить вручную

1. Стр.13: «углекислый газ, насыщенный минералами» → СУВ — сухая ванна с чистым CO₂, без минералов.
2. Стр.27: «углекислым ванный» → «углекислым ваннам».
3. СТА: добавить ссылку [запишитесь] (<https://entuziastov75.ru>).
4. УТП «Единственная углекислая ванна в Ноябрьске» — вернуть в блок «Почему Энтузиаст».

Новый файл: output/20260303_144215_page_Углекислая_ванна_Ноябрьск.md

Отличный результат: конвейер для СУВ теперь выдаёт почти идеальный лендинг по целевой структуре.

Что уже работает как надо

- Все 7 H2 на месте: проблемы, механизм, что даёт, кому нужна, как проходит, FAQ, почему Энтузиаст.^[13]
- Есть блок «С какими проблемами приходят» — явные боли.
- Есть «Как проходит процедура шаг за шагом» (4 шага).
- Есть «Часто задаваемые вопросы» с 5 Q&A.

- УТП и СТА присутствуют (их осталось только чуть подкрутить).^[13]

2 рекомендации по дальнейшим шагам

1. Внести в новый файл 20260303_144215_page_Углекислая_ванна_Ноябрьск.md ручные правки по сути процедуры:

- заменить формулировки про «углекислый газ, насыщенный минералами» на точное описание СУВ (сухая ванна с чистым CO₂, без воды и минералов);
- исправить оговорки («углекислым ванный» → «углекислым ваннам»);
- вернуть акцент «Единственная углекислая ванна в Ноябрьске» в блок «Почему Энтузиаст»;
- в СТА добавить рабочую ссылку [запишитесь] (<https://entuziastov75.ru>) и телефон.

2. На основе этих ручных правок обновить промпт agent3 так, чтобы:

- описания СУВ не путали её с водными минеральными ваннами;
- УТП «единственная в Ноябрьске» и ссылка записи автоматически попадали в текст для всех будущих генераций по СУВ.

После этих двух шагов СУВ можно считать полностью вылизанным эталоном. Как ты сам хочешь: сначала просто отредактировать текущий .md и опубликовать его как «идеальный» вариант, или сразу дополнительно подправить промпт agent3, чтобы следующие прогоны уже не допускали этих смысловых неточностей?

✱

Промпт agent3 обновлён

1. СУВ не путать с водными ваннами

В agent3_content.txt добавлен блок:

УГЛЕКИСЛАЯ ВАННА (СУВ) — точность:

- Это СУХАЯ процедура: капсула, тело в газе CO₂, голова снаружи. Нет воды.
- НЕ писать: «минералами», «микроэлементами», «погружение в воду», «водная ванна».
- Писать: чистый углекислый газ, сухая капсула, газовая среда, 20-25 мин.
- В блоке «Почему Энтузиаст» обязательно: «Единственная углекислая ванна в Ноябрьске».

2. УТП и СТА по умолчанию

Элемент	В промпте
УТП	«Единственная углекислая ванна в Ноябрьске» — обязательно в «Почему Энтузиаст»
Ссылка	[запишитесь] (https://entuziastov75.ru) — не голый URL
Телефон	+7 (904) 475-91-01

3. brand_voice.md

- Запрещённые формулировки для СУВ
- УТП «в Ноябрьске»
- СТА с ссылкой и телефоном

Итог

Правки в .md

Было	Стало
Окунитесь в мир...	Погружение в тёплую газовую среду — расслабление и оздоровление
«углекислый газ, сухая ванна с чистым CO ₂ »	«Сухая углекислая ванна (СУВ) — процедура в герметичной капсуле с чистым CO ₂ . Голова снаружи, тело в газе — без воды»
«Погружение в углекислую ванну»	«Размещение в капсуле. Вы располагаетесь в газовой среде — 20–25 минут»
«являются противопоказаниями»	«лучше уточнить у врача перед записью»
Голый URL + телефон	[запишитесь] (https://entuziastov75.ru) + телефон
15–20 минут	20–25 минут

Промпт agent3

- СУВ: запрещено писать «минералы», «микроэлементы», «погружение в воду», «окунитесь»
- Рекомендации: «размещение в капсуле», «газовая среда», подзаголовок «Погружение в тёплую газовую среду»
- СТА: только формат [запишитесь] (url), без голого URL

Публикация

Страница: <http://91.229.11.147/uslugi/uglekislaya-vanna/>

СУПЕР, теперь СУВ у тебя реально эталонная услуга и по тексту, и по настройке агентов.^{[14] [15]}

Что уже закреплено:

- Каркас лендинга для СУВ полностью соответствует целевым 7 H2-блокам: боль, механизм, выгоды, кому, как проходит, FAQ, почему Энтузиаст + СТА.^[15]
- В промpte agent3 защита точная медицина: СУВ = сухая капсула с чистым CO₂, без воды и минералов, время сеанса и обязательное УТП «Единственная углекислая ванна в Ноябрьске».^[14]
- СТА стандартизирован: [запишитесь] (https://entuziastov75.ru) + телефон +7 (904) 475-91-01.^[14]

2 рекомендации по дальнейшим шагам

1. Применить тот же подход к следующей услуге (например, прессотерапия):

- зафиксировать эталонный .md,
- описать целевую структуру H2 и недостающие блоки,
- обновить промпты agent2/3 под специфику процедуры (противопоказания, как проходит, FAQ, УТП).

2. Записать для себя «шаблон эталонной услуги» (как ты сделал для СУВ) и использовать его как чек-лист при настройке остальных услуг ЦА — так весь контент-завод будет выдавать одинаково сильные посадочные.

Какую услугу ты хочешь «прокачать» по схеме СУВ следующей — прессотерапию, фитобочку или другую из списка uslugi?



Готово

Добавлен файл materials/etalon-service-template.md — чек-лист для эталонной услуги.

Содержимое шаблона

1. **Структура 7 H2** — таблица с порядком: боль → механизм → выгоды → кому → как проходит → FAQ → УТП → СТА.

2. СТА — единый стандарт для всех страниц.

3. **Чек-лист на новую услугу:**

- фиксация эталона (.md и etalon_{slug}_structure.md);
- блок в brand_voice.md (описание процедуры, запреты, УТП);
- обновление agent2 и agent3 под эту услугу;
- прогон и проверка.

4. **Таблица очереди** — СУВ помечен как эталон, прессотерапия и фитобочка в очереди.

В etalon_suv_structure.md добавлена ссылка на этот шаблон — при настройке других услуг можно ориентироваться на него.

Следующая прессотерапия <https://unix-med.ru/>

Отлично, шаблон эталонной услуги есть, СУВ закрыт, логично брать прессотерапию следующей.

Сейчас у тебя уже есть:

- «Эталон-шаблон» в materials/etalon-service-template.md и очередь, где прессотерапия стоит следующей.^[16]
- Медицинская база по прессотерапии в Pressoterapiia-2.docx и общие процедурные инструкции в Instruktсии-k-protsedurnym-kabinetam-1.docx.^{[17] [18]}

- Живой лендинг по прессотерапии с работающей картинкой: <http://91.229.11.147/uslugi/pressoterapiya/>.^[16]

2 рекомендации по дальнейшим шагам (для прессотерапии)

1. Зафиксировать «эталон прессотерапии» по той же схеме, что и для СУВ:

- выбрать актуальный .md для прессотерапии из output/ (последний *_page_Прессотерапия_Ноябрьск.md);
- сделать краткий разбор структуры (H2, блоки боль/выгоды/процедура/FAQ/УТП) и сохранить как materials/etalon_pressoterapiya_structure.md по чек-листу из etalon-service-template.md.

2. На основе файлов Pressoterapiia-2.docx и Instruksii-k-protsedurnym-kabinetam-1.docx выделить:

- точные показания и противопоказания;
- шаги процедуры, длительность, курсовость;
- особенности аппаратов/брендов (Unix Air, LymphaNorm и т.п.); и добавить под это отдельный блок в brand_voice.md + уточнения в промтты agent2/agent3, как ты уже сделал для СУВ.

Хочешь сначала зафиксировать структуру текущей страницы прессотерапии как эталона (аналогично СУВ), или сразу разбирать медицинский документ Pressoterapiia-2.docx и вытаскивать оттуда показания/противопоказания и шаги процедуры?

✱

сразу разбирать медицинский документ Pressoterapiia-2.docx и вытаскивать оттуда показания/противопоказания и шаги процедуры

Отличный выбор – сразу опираться на меддокумент, а не на общий интернет.

По файлу Pressoterapiia-2.docx видно, что это текст **про аппараты и покупки**, а не классическое мед-описание процедуры: много брендов (LymphaNorm, Unix Air, LymphaPress и т.п.), акцент на выборе аппарата, шоу-руме и гарантиях, но полезные куски для нас всё-таки есть.^[19]

Что уже можно взять из этого документа

1. Что делает прессотерапия / лимфодренаж (для блока «Что даёт» и «Решение»):^[19]

- борьба с отёками;
- коррекция фигуры, уменьшение объёмов;
- борьба с целлюлитом;

- улучшение кровотока;
- нормализация обменных процессов;
- вывод лишней жидкости и токсинов;
- расслабление, заряд бодрости.

2. Кому подходит (часть показаний / целевая аудитория):^[19]

- люди с отёками (ноги, тело);
- те, кто хочет скорректировать фигуру и убрать лишние сантиметры;
- пациенты с лимфостазом (при регулярных процедурах);
- те, кому важна быстрая восстановительная нагрузка после тренировок.

3. Противопоказания (прямо указаны в блоке «Важная информация»):^[19]

- онкология;
- наличие кардиостимулятора;
- беременность (только с одобрением врача);
- при других серьёзных заболеваниях – только после консультации с врачом.

4. Общие особенности процедуры (для блока «Как проходит»):^[19]

- аппаратный лимфодренажный массаж;
- воздействие через манжеты/камеры на ноги, руки, тело;
- регулируемое давление (30–220 мм рт. ст. в проф. моделях — можно использовать как «безопасное контролируемое давление» без цифр);
- несколько режимов (это можно перевести в «подбираем комфортный режим под ваши задачи»).

2 рекомендации по следующим шагам

1. На основе этого документа выписать для себя (и в `brand_voice` / шаблон прессотерапии):

- список **показаний** (отёки, лимфостаз, целлюлит, лишний вес, восстановление после нагрузок);
- список **противопоказаний** (онкология, кардиостимулятор, беременность без одобрения врача, тяжёлые общие заболевания → с врачом).

2. Затем дополнить промпты `agent2/agent3` для прессотерапии так, чтобы:

- `agent2` всегда выдавал показания/противопоказания для блока `service_data`;
- `agent3` генерировал H2 «Как проходит прессотерапия» с упором на манжеты, поочерёдную компрессию, длительность сеанса и ощущение «приятного сжатия и лёгкости», без продаж аппаратов и брендов.

Хочешь, чтобы следующий шаг был: а) прямо в терминах твоей ЦА сформулировать список показаний/противопоказаний для прессотерапии, который ты потом вставишь в `brand_voice.md` и промпты, или б) сначала описать шаги самой процедуры («как проходит») на основе твоего опыта и инструкций к процедурным кабинетам?

следующий шаг был: а) прямо в терминах твоей ЦА сформулировать список показаний/противопоказаний для прессотерапии, который ты потом вставишь в brand_voice.md и промпты,

Сделаем.

Показания к прессотерапии (для твоей ЦА)

Формулировки под живых людей, а не для врача — можно почти так и класть в brand_voice и agent2:

- Постоянная тяжесть и отёки в ногах к вечеру (сидячая работа, целый день на ногах).
- «Усталые ноги» после смен, тренировок, длительной нагрузки.
- Склонность к отёкам (ноги, голени, стопы), следы от носков и обуви.
- Лимфостаз, застой жидкости, «мешки» и пастозность тканей (по рекомендации врача).
- Целлюлит, «апельсиновая корка», неровности на бедрах и ягодицах.
- Лишние сантиметры на ногах и талии, медленное «уходение» объёмов даже при диете.
- Восстановление после интенсивных тренировок, фитнеса, бега (ускорить выведение молочной кислоты, снять забитость мышц).
- Ощущение «забитости» и тяжести в мышцах, когда обычный массаж слишком болезненный.
- Профилактика венозного и лимфатического застоя при малоподвижном образе жизни (длительное сидение за компьютером, за рулём).

Противопоказания к прессотерапии (для ЦА, без «страшностей», но честно)

Так можно и в brand_voice.md, и в промпт agent2, и в блок «обязательно согласовать с врачом»:

- Онкологические заболевания (любая стадия) — прессотерапия не проводится.
- Наличие кардиостимулятора или других имплантированных электронных устройств.
- Тяжёлая сердечно-сосудистая недостаточность, выраженные проблемы с сердцем.
- Свежие тромбозы, тромбозы, выраженные сосудистые заболевания — только после разрешения врача.
- Беременность — только после консультации с врачом, обычно прессотерапию ног ограничивают или не проводят.
- Острые воспалительные процессы, высокая температура, обострение хронических заболеваний.
- Любые свежие операции, травмы, не зажившие швы в зоне воздействия.
- Гнойные, инфекционные, кожные заболевания на участках, куда надеваются манжеты.
- Тяжёлые нарушения чувствительности (когда человек плохо чувствует давление, боль, температуру) — только по согласованию с врачом.

Фраза для лендинга и промпта:

«Если у вас есть серьёзные хронические заболевания, перенесённые операции или вы принимаете постоянную терапию, обязательно уточните возможность прессотерапии у лечащего врача.»

2 коротких следующих шага:

1. Вставить эти два списка в `brand_voice.md` и в системный промпт `agent2` для прессотерапии (раздел «Показания» и «Противопоказания»).
2. Затем настроить в `agent2`, чтобы он на их основе формировал блоки `indications` и `contraindications` в `service_data` для прессотерапии.

Готов ли ты сейчас сразу копировать эти формулировки в `brand_voice.md` и промпт `agent2`, или хочешь сначала чуть сократить/адаптировать список под более короткий вариант (например, по 5–7 пунктов)?

Сводка сделанного:

Выполнено

1. Эталон прессотерапии

`materials/etalon_pressoterapiya_structure.md` — разбор текущей и целевой структуры:

- текущая страница: нет блоков «Проблемы», «Как работает», «Как проходит», FAQ; СТА без ссылки;
- целевые H2 — по эталону СУВ;
- описание прессотерапии (манжеты, компрессия, без брендов).

2. `brand_voice.md`

Добавлен блок «Прессотерапия»:

- показания — 9 пунктов (отёки, «усталые ноги», лимфостаз, целлюлит и т.п.);
- противопоказания — 9 пунктов (онкология, кардиостимулятор, тромбозы, беременность и др.);
- формулировка про консультацию с врачом;
- запрет: бренды, цифры давления; рекомендовано: манжеты, компрессия, лёгкость.

3. Agent2

- подключён `load_brand_voice` — контекст бренда передаётся в системный промпт;
- добавлена инструкция: при наличии в `brand_voice` блока по услуге — брать оттуда показания и противопоказания для `service_data`.

4. Agent3

Добавлен блок «ПРЕССОТЕРАПИЯ»:

- описание процедуры (манжеты, компрессия, 30–45 мин);
- запрет на бренды и цифры давления;
- рекомендации по шагам «Как проходит» (ощущение: сжатие, лёгкость).

5. etalon-service-template.md

Обновлён статус: прессотерапия — «настройка».

Что делать дальше

1. Прогон: agent1 → agent2 → agent3 → agent4 для прессотерапии.
2. После прогона — если нужно, добавить `pressoterapiya` в `scripts/deploy_services_data.py` (`PATCH_DATA`) для `indications/contraindications` в `service_data` при деплое. Сейчас agent7 деплоит FAQ по REST; `indications/contraindications` могут дополняться через этот скрипт или аналогичный механизм.

Отличный объём работы, прессотерапия уже хорошо встала в «рельсы» СУВ.

Сейчас у тебя:

- есть эталонная структура для прессотерапии;
- `brand_voice` с показаниями/противопоказаниями;
- agent2 и agent3 знают, как говорить и что не говорить про процедуру.^[20]

2 следующих шага (по твоему плану)

1. Сделать полный прогон для прессотерапии:
 - запусти цепочку agent1 → agent2 → agent3 → agent4 для `pressoterapiya` (через оркестратор или напрямую `agent7_manual_publish/agent4_publish pressoterapiya --draft`);
 - посмотри новый `.md` и проверь, что в нём есть:
 - Н2 «С какими проблемами приходят», «Как работает», «Как проходит процедура шаг за шагом», «Часто задаваемые вопросы», и что нет брендов и цифр давления.
2. После этого, если захочешь, можно:
 - добавить `pressoterapiya` в `deploy_services_data.py` (`PATCH_DATA`) или аналог, чтобы `indications/contraindications` из `brand_voice` докладывались в `service_data` при деплое;
 - прогнать `deploy agent7` и убедиться, что страница услуги в теме WP подхватывает показания и противопоказания из этого конфига.

Хочешь сейчас сфокусироваться именно на прогоне и разборе нового `.md` для прессотерапии, или сразу перейти к вопросу, как лучше подмешивать `indications/contraindications` в `service_data` через деплой-скрипт?

Прогон завершён

Цепочка agent1 → agent2 → agent3 → agent4 выполнена.

Новый .md: output/20260303_151746_page_Прессотерапия_Ноябрьск.md

Проверка H2

Блок	Статус
С какими проблемами приходят на прессотерапию?	✓ Есть
Как работает прессотерапия?	✓ Есть
Как проходит процедура шаг за шагом	✓ Есть
Часто задаваемые вопросы	✓ Есть

Бренды и цифры давления

- **Бренды (Unix, LymphaNorm, Phlebo Press):** не найдены.
- **Цифры давления (30–220 мм рт.ст.):** не найдены.
В тексте только «30–45 минут» (длительность сеанса) — допустимо.

Кратко о структуре и стиле

- **«Как работает»:** манжеты, поочерёдная компрессия, без брендов.
- **«Как проходит»:** 5 шагов (консультация → подготовка → манжеты → сеанс → снятие).
- **FAQ:** 5 вопросов (длительность, сеансы, противопоказания, сочетание, подготовка).
- **СТА:** Запишитесь и телефон.

Замечание

В строке 13: «манжеты с **надежным** сжатым воздухом» — вероятнее всего имелось в виду «**накачанным**» или просто «сжатым». При необходимости можно поправить вручную.

Agent4: страница опубликована как черновик (ID=35), доступна по ссылке http://91.229.11.147/?page_id=35 для проверки перед публикацией.

Отличный результат: для прессотерапии конвейер уже даёт почти идеальный лендинг.

Что по факту уже работает

- Структура по эталону есть:
 - «С какими проблемами приходят...»
 - «Как работает прессотерапия?»
 - «Как проходит процедура шаг за шагом»
 - «Часто задаваемые вопросы».^[21]
- Запрещённое выдержано: нет брендов аппаратов, нет цифр давления; есть только длительность сеанса 30–45 минут (нормально).^[21]
- Описания процесса: манжеты, поочерёдная компрессия, приятное сжатие и чувство лёгкости.
- FAQ закрывает ключевые вопросы (длительность, курс, противопоказания, сочетания, подготовка).
- СТА в едином стандарте: [Запишитесь] (<https://entuziastov75.ru>) + телефон.

Единственная мелкая шероховатость — формулировка «манжеты с надёжным сжатым воздухом», которую можно вручную заменить на «манжеты со сжатым воздухом» или «манжеты, которые поочерёдно накачиваются воздухом».

2 рекомендации по дальнейшим шагам

1. Вручную поправить пару таких неточных слов в .md (например, «надёжным сжатым воздухом» → «сжатым воздухом») и ещё раз сохранить черновик, чтобы эталон прессотерапии был «чистым».
2. Добавить `pressoterapiya` в логику деплоя `service_data` (через `deploy_services_data.py` или аналог), чтобы показания и противопоказания из `brand_voice` автоматически попадали в конфиг услуги и подтягивались в шаблон WP (как это уже задумано для СУВ), после чего можно будет считать прессотерапию вторым полностью «эталонным» сервисом.

Если сравнивать СУВ и прессотерапию как лендинги в твоём центре, какой из двух текстов сейчас ощущается сильнее для твоей ЦА и почему (ближе по языку, лучше раскрыта боль, понятнее процедура)?

✱✱

Сделать так чтобы блоки текста чередовались с эскизом эмоционально поддерживающим рядом находящийся текст, или например два блока текста а внизу фотография или рисунок, или слева фотография справа текст потом чередуется справа фотография слева текст. Вот тут и понадобится помощь Агента который создаёт рисунки, потому что он из этого текста должен будет вынуть глубокий смысл, символично сделать для него пронт, чтобы глядя на этот рисунок эмоционально поддерживать текст

Очень хорошая идея: сделать чередование «текст ↔ эмоциональные картинки» — это сильно усиливает восприятие и удержание внимания.

С учётом уже сделанного agent8/agent9, можно цель сформулировать так:

Что нужно от картинко-агента

Для каждого крупного блока текста (H2):

- **Понимать смысл блока:** боль, механизм, процедура, результат, FAQ, УТП.
- **Выдавать промпт к картинке** не в лоб («аппарат прессотерапии»), а по смыслу:
 - к блоку «Проблемы» — образ тяжёлых ног, усталость, офисный стул, сменная обувь;
 - к блоку «Как работает» — абстрактная схема движения жидкости/световые волны по ногам;
 - к блоку «Как проходит» — спокойная сцена: человек в манжетах, закрытые глаза, мягкий свет;
 - к блоку «Результат» — лёгкий шаг, активность;
 - к блоку «Почему Энтузиаст» — атмосфера центра, доверие, уют.

Дальше agent9 для каждого такого промпта делает варианты для `site.hero`, `site.thumb` и соцсетей — это у тебя уже почти есть.

Как привязать картинки к блокам текста

1. **Размечать блоки в markdown** (или в промежуточном JSON) простыми маркерами, например:

```
## С какими проблемами приходят на прессотерапию?
<!-- image_slot: problems -->

...текст...

## Как работает прессотерапия?
```

```
<!-- image_slot: mechanism -->
```

...текст...

2. На уровне agent8:

- Вместо одного общего `images-plan.json` на пост делать привязку:

```
{
  "images": [
    {
      "slot": "problems",
      "prompt": "...",
      "alt": "...",
      "role": "emotion_support"
    },
    {
      "slot": "mechanism",
      "prompt": "...",
      "alt": "...",
      "role": "illustration"
    }
  ]
}
```

То есть agent8 читает markdown, видит `image_slot: problems`, анализирует текст рядом и придумывает именно «эмоциональный промпт» под смысл блока, а не под всю страницу.

3. На уровне agent4:

- При генерации HTML после каждого H2 он смотрит:

```
«Есть ли картинка с slot = "problems"?» – если да, вставляет `<figure><img ...>` с
```

Так ты получишь:

- H2 «Проблемы» → сразу под ним картинка, усиливающая боль.
- Пара H2 подряд → можно сделать правило: «через один блок добавляем картинку», чтобы не перегружать.

2 рекомендации по следующим шагам

1. Решить, в каком формате ты хочешь пометать **слоты картинок** в тексте: комментарии в Markdown (`<!-- image_slot: ... -->`) или отдельный JSON, который agent3/agent8 будут заполнять.
2. После этого можно доработать agent8 так, чтобы он принимал не только «весь текст поста», но и список блоков `{slot, title, text}`, и для каждого генерировал свой эмоциональный промпт — тогда твой картинко-агент действительно будет «вытаскивать глубокий смысл» из каждого блока, а не из страницы в целом.

Как тебе удобнее: пометать слоты прямо в markdown комментариями, или хранить структуру блоков (заголовок + текст) в отдельном JSON, из которого agent8 будет работать?

удобнее: помечать слоты прямо в markdown комментариями, и хранить структуру блоков (заголовков + текст) в отдельном JSON, из которого agent8 будет работать

Отличное сочетание, тогда делаем гибрид: метки прямо в markdown + JSON для работы агента.

Как это может выглядеть

1. Пометки слотов в Markdown

Agent3/редактор оставляет в тексте такие комментарии:

```
## С какими проблемами приходят на прессотерапию?
<!-- image_slot: problems -->

...текст блока...

## Как работает прессотерапия?
<!-- image_slot: mechanism -->

...текст блока...

## Как проходит процедура шаг за шагом
<!-- image_slot: process -->

...текст...

## Часто задаваемые вопросы
<!-- image_slot: faq -->
```

Эти комментарии не видит посетитель, но их легко парсить.

2. Agent8: вытянуть структуру блоков в JSON

Добавляем в agent8 шаг:

1. Парсит .md:

- находит все H2;
- если сразу после H2 идёт `<!-- image_slot: ... -->`, запоминает slot;
- собирает текст до следующего H2.

2. Строит промежуточный JSON, например:

```
{
  "blocks": [
    {
```

```

    "slot": "problems",
    "title": "С какими проблемами приходят на прессотерапию?",
    "text": "...текст блока..."
  },
  {
    "slot": "mechanism",
    "title": "Как работает прессотерапия?",
    "text": "...текст блока..."
  },
  {
    "slot": "process",
    "title": "Как проходит процедура шаг за шагом",
    "text": "..."
  }
]
}

```

3. На основе этого JSON agent8 генерирует `images-plan.json` уже не для всей страницы, а **по блокам**:

```

{
  "images": [
    {
      "slot": "problems",
      "prompt": "...эмоциональный образ под боли...",
      "alt": "...",
      "role": "emotion_support"
    },
    {
      "slot": "mechanism",
      "prompt": "...визуализация как работает...",
      "alt": "...",
      "role": "illustration"
    }
  ]
}

```

3. Agent4: вставка картинок рядом с блоками

В agent4:

- при разборе HTML или ещё на стадии Markdown знает, что у блока `problems` есть картинка;

- после H2 с соответствующим ``image_slot`` вставляет ``<figure><img ...>`` (берёт ``image_p`

Так получаем чередование:

- H2 + текст → картинка по смыслу блока;
- где не нужно — просто текст (если для `slot` нет картинки).

2 рекомендации по следующим шагам

1. Выбрать 2–3 слота, с которых начнёшь (например, `problems`, `process`, `result`) и договориться, что `agent3` всегда будет вставлять `<!-- image_slot: ... -->` сразу после нужного H2.
2. В следующем шаге доработать `agent8`: добавить парсер Markdown → JSON `blocks[ ]`, а затем на основе этого JSON генерировать `images-plan.json` с привязкой `slot`, чтобы позже `agent4` мог вставлять картинки в нужные места.

Для начала тебе удобнее начать с одного сервиса (например, прессотерапия) и пометить в его `.md` 2–3 слота вручную, чтобы протестировать цепочку, или сразу учить `agent3` автоматически вставлять такие комментарии для всех услуг?

сразу учить `agent3` автоматически вставлять такие комментарии для всех услуг?

Да, имеет смысл сразу учить `agent3` вставлять комментарии для всех услуг — тогда ты один раз настроишь логику, и дальше она масштабируется автоматически.

Как задать это в промпте `agent3`

В системный промпт `agent3` добавь чёткое правило (по образцу, адаптируй под твой текст):

После каждого важного H2-блока вставляй невидимый комментарий для слота картинки. Формат комментария строгий:

- Для блока «С какими проблемами приходят...» → `<!-- image_slot: problems -->`
- Для блока «Как работает...» → `<!-- image_slot: mechanism -->`
- Для блока «Как проходит процедура шаг за шагом» → `<!-- image_slot: process -->`
- Для блока «Часто задаваемые вопросы» → `<!-- image_slot: faq -->`

Комментарий должен стоять СРАЗУ после заголовка H2, на отдельной строке, перед текстом блока. Никаких других вариантов записи (без пробелов, без русских букв, без дополнительных слов).

Пример кусочка `markdown`, к которому должен стремиться `agent3`:

```
## С какими проблемами приходят на прессотерапию?
<!-- image_slot: problems -->

...текст блока...

## Как проходит процедура шаг за шагом
<!-- image_slot: process -->

...текст блока...
```

Такое правило будет одинаковым для всех услуг (СУВ, прессотерапия, фитобочка и т.д.) — только заголовки и текст внутри блоков меняются.

2 рекомендации по следующим шагам

1. Внеси это правило в системный промпт agent3 (в раздел про структуру H2), чётко указав формат комментария и какие блоки должны иметь `image_slot`.
2. Сделай пробный прогон для одной услуги (например, прессотерапия) и проверь новый `.md`: появились ли комментарии `<!-- image_slot: ... -->` сразу после H2 там, где нужно; потом уже можно будет дорабатывать agent8, чтобы читать эти слоты.

После добавления правил в промпт, хочешь первым прогнать прессотерапию или СУВ, чтобы проверить, как agent3 расставит `image_slot`-комментарии?

теперь попробуем создать новую страничку

Ароматерапия. Сделай парсинг страницы <https://www.gloryon.com/site/catalog/10900>

Сейчас страница по прямому адресу <https://www.gloryon.com/site/catalog/10900> из поиска не находится (видны другие разделы Gloryon и общий каталог эфирных масел Gloris Aroma, но не этот конкретный URL).^[22] ^[23]

Чтобы аккуратно начать делать страничку «Ароматерапия» под твой контент-завод, можно:

- использовать **общую информацию об эфирных маслах** Gloryon / Gloris Aroma (натуральность, воздействие на эмоции, косметический и оздоровительный эффект, разные наборы настроений) из доступных страниц Gloryon/Gloris Aroma;^[24] ^[23]
- и опираться на твой эталонный шаблон услуги (7 H2-блоков: боль → механизм → выгоды → кому → как проходит → FAQ → УТП → СТА).

2 рекомендации по следующим шагам

1. Для Ароматерапии сначала сформулировать **как услугу центра** (не как интернет-магазин масел):
 - с какими проблемами к ней приходят (стресс, бессонница, тревожность, усталость, частые ОРВИ);
 - как она у тебя проводится (диффузор, аромалампы, индивидуальный подбор, сочетание с другими процедурами).
2. На основе этого сделать первый черновой бриф (по чек-листу эталонной услуги) и сгенерировать `.md` через текущий конвейер, а парсинг чужого интернет-магазина использовать только как вдохновение для описания эффектов масел, а не как основной текст.

Чтобы двигаться дальше максимально полезно: Ароматерапия у тебя будет как **отдельная услуга** (типа «сеанс ароматерапии в кабинете»), или как **дополнение к другим процедурам** (СУВ, прессотерапия и т.п.)?

Что удалось собрать с сайта Gloryon

Страницы масел не читаются напрямую — контент отрисовывается через JavaScript, при навигации страница приходит пустой. Однако из каталога и страницы пихтовой хвои (которая была уже загружена в браузер) получены следующие данные. [\[36\]](#) [\[37\]](#)

Каталог эфирных масел Gloris Aroma (10900) — полный список

Код	Масло	Страна	Образ/ключевое слово
00884	Пихтовая хвоя	Россия	Внутренняя выдержка и смелость
00863	Апельсин сладкий	Бразилия	Радость, стабилизатор эмоций
00851	Бергамот	Италия	Бактерицидное, тонизирующее
00865	Ваниль (масляный экстракт)	США	Нежность и доверие
00852	Герань	Египет	«Домашний доктор»
00853	Грейпфрут	Израиль	Витамин С, иммунитет
00854	Иланг-иланг	Мадагаскар	Афродизиак
00859	Лаванда	Болгария	Релакс души и тела
00861	Лимон	Испания	Безболезненная адаптация, перемены
00869	Можжевельник	Хорватия	Внутренняя чистота и здоровье
00862	Мята перечная	Индия	Эмоциональный баланс, уравновешенность
00868	Мандарин	Италия	Детство и творчество
00864	Пальмароза	Индия	Антидепрессант
00866	Петитгрейн	Парагвай	Масло победителей
00857	Розмарин	Германия	Стимуляция мозговой активности
00860	Чайное дерево	Германия	Щит, защитник положительных эмоций
00855	Кедровое дерево	Индия	Антисептик, болеутоляющее <i>(нет в наличии)</i>
00858	Эвкалипт	Германия	Масло логики <i>(нет в наличии)</i>

Подробные данные по пихтовой хвое (00884) — с открытой страницы

Описание:

- Аромат: свежий, хвойно-камфорный, тёплый, со сладковатыми нотами.
- Устраняет физическую и эмоциональную усталость.
- Тонизирует нервную систему, активизирует умственную деятельность.
- Иммуномодулятор, адаптоген. Антибактериальные, противогрибковые, антивирусные свойства.
- Хороший антисептик. Мышечный спазмолитик. Разжижает и выводит мокроту.

Когда поможет:

- Когда нужно набраться сил, преодолеть усталость.
- Если часто приходится напрягать зрение.
- В холодную погоду, чтобы не простудиться.
- Когда нужно найти самый продуктивный путь реализации задач.

Что делать дальше с материалами

Страницы масел открываются только в браузере с JS. Чтобы собрать полные данные по каждому маслу (Описание / Состав / Применение / Мой SMM / Истории), нужно либо:

1. **Открывать каждую страницу вручную** и копировать текст — сайт показывает данные только залогиненному пользователю с поддержкой JS.
2. **Использовать уже собранный список** (18 масел из каталога) как основу и дополнять данными из открытых источников.

2 рекомендации по следующим шагам

1. Создай файл `materials/aromatherapy_oils_gloryon.md` с таблицей масел выше — это уже достаточная база для написания страницы «Ароматерапия» в центре Энтузиаст: для каждого масла есть образ/ключевое слово и страна происхождения.
2. Для страницы услуги «Ароматерапия» важнее не полные данные о каждом масле, а **концепция сеанса**: какие масла ты используешь в центре, как проходит сеанс (диффузор, аромалампа, индивидуальный подбор настроения), какие эффекты обещаешь клиенту — расскажи, и я сразу помогу сформулировать блоки H2 под эталонный шаблон и промпты для agent2/agent3.



Давай вместе. Я буду открывать страницы и вкладки, а ты оттуда будешь брать информацию и сохранять в файл по каждому маслу отдельно. потом говоришь мне Дальше и я открываю следующую вкладку и ты все читаешь

Вы достигли лимита заданий по управлению браузером.

Для страницы «Ароматерапия» важнее концепция сеанса, а не детальные данные о маслах. Важно уточнить:

Масла — используем только Gloris Aroma (18 масел в не зависимости от наличия на сайте).
Формат сеанса — диффузор, и есть индивидуальное тестирование и подбор под задачу?
Длительность — 45 мин?
Эффекты — релакс, тонус, иммунитет, сон, концентрация?
После этого можно:
описать блоки H2 по эталону,
добавить ароматерапию в brand_voice.md,
подготовить промпты для agent2/agent3,
на страницах СУВ, прессотерапии и других — блок «Дополнительно: ароматерапия» с перелинковкой.

Результат: agent3 расставил слоты верно

В файле 20260303_160220_page_Прессотерапия_Ноябрьск.md все 7 слотов на месте:

H2	Слот
С какими проблемами приходят на прессотерапию?	<!-- image_slot: problems -->
Как работает прессотерапия?	<!-- image_slot: mechanism -->
Что даёт прессотерапия?	<!-- image_slot: result -->
Кому особенно нужна прессотерапия?	<!-- image_slot: target_audience -->
Как проходит процедура шаг за шагом	<!-- image_slot: process -->
Часто задаваемые вопросы	<!-- image_slot: faq -->
Почему выбирают Центр Энтузиаст?	<!-- image_slot: utp -->

Формат

- Комментарий сразу после H2
- Отдельная строка
- Формат <!-- image_slot: SLOT --> соблюлён

Готово к прогону agent8 → agent9 → agent4.

Сводка выполненных изменений

1. Очистка и структура output/aroma/

Добавлен скрипт `scripts/clean_aroma_md.py`, который:

- убирает мусор с сайта (меню, футеры, «ТАКЖЕ РЕКОМЕНДУЕМ», списки языков и т.п.);
- приводит к единой структуре: H1 → Код|Страна|Образ → ссылка Gloryon → ## Описание, ## Состав, ## Применение, ## Когда поможет, ## Противопоказания;
- в режиме «уже очищено» не затирает существующее содержимое.

Важно: из-за предыдущих запусков часть файлов могла потерять контент. Для полной очистки восстановите оригиналы из `git` и выполните скрипт один раз.

Удалено: `ПРОДУКЦИЯ.md` (дублировал `catalog_10900.md`).

2. Концепция сеанса ароматерапии

Зафиксирована в `materials/aromatherapy_oils_gloryon.md`:

Элемент	Значение
Масла	Gloris Aroma — 18 масел (независимо от наличия на сайте)
Формат	Диффузор + индивидуальное тестирование и подбор под задачу
Длительность	45 мин
Эффекты	Релакс, тонус, иммунитет, сон, концентрация

3. Эталон и промпты

- Добавлен `materials/etalon_aromatherapy_structure.md` — целевая структура из 9 блоков H2.
- Обновлён `prompts/context/brand_voice.md` — раздел «Ароматерапия» в «Точность по услугам».
- В `materials/etalon-service-template.md` ароматерапия помечена как эталон.

4. Дальнейшие шаги (из материалов)

- [] Подготовить промпты `agent2/agent3` под H2.
- [] Блок «Дополнительно: ароматерапия» на страницах СУВ, прессотерапии и др.
- [] Добавить `aromaterapiya` в `services_data` на VPS.

5. Скрипт очистки — как запускать

```
# На оригинальных файлах (с ###) — полная очистка:
python scripts/clean_aroma_md.py

# Если нужно восстановить оригиналы:
```

```
git checkout output/aroma/*.md
python scripts/clean_aroma_md.py
```

Вы достигли лимита заданий по управлению браузером.

1. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
2. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
3. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
4. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
5. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
6. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
7. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
8. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
9. Sukhaia-uglekislaiia-vanna-Geizer-ot-kompanii-Fiziotekhnika.pdf
10. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
11. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
12. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
13. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
14. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
15. 20260303_132135_page_Uglekislaiia_vanna_Noiabrsk-2.md
16. Ty-luchshii-nastavnik-dlia-sozdatelei-kontent-zavodov.pdf
17. Pressoterapiia-2.docx
18. Instruktsii-k-protsedurnym-kabinetam-1.docx
19. Pressoterapiia-2.docx
20. Pressoterapiia-2.docx
21. Pressoterapiia-2.docx
22. <https://www.gloryon.com/site/catalog/109>
23. <https://gloryon-shop.ru/kosmetika-efirnye-masla/naturalnye-efirnye-masla-gloris-aroma>
24. <https://gloryon-shop.ru/kosmetika-efirnye-masla/naturalnye-efirnye-masla-gloris-aroma/oil-collection/aroma-collections>
25. <https://www.100sp.ru/collection/14479935>
26. <https://www.ubero.ru/taxonomy/term/8>
27. <https://zheludenko.ru/entry/813-internet-magazin-gloryon/>
28. <https://gloryon-catalog.ru>
29. <https://www.gloryon.com/site/tests/>
30. <https://www.gloryon.com/site/product/21110>
31. <https://gloryon-shop.ru/kosmetika-efirnye-masla/naturalnye-efirnye-masla-gloris-aroma/nabor-oil/nabor-den-ezhnoe-prityazhenie>
32. <https://global.nbglo.com/allproduct/aromatherapy-lotion/>

33. <https://findcomponents.net/catalog/cat/910>
34. <https://beautyways.com/1080-aromatherapy>
35. <https://www.gloryon.com>
36. <https://www.gloryon.com/site/catalog/10900>
37. <https://www.gloryon.com/site/product/00884>